

A Practical MPC-Pretrained Reinforcement Learning Framework for Complex Process Systems

Amir Hossein Hamed, Hesam Hassanpour, Ankur Kumar, Atharva Vijay Suryavanshi, and Prashant Mhaskar*



Cite This: *Ind. Eng. Chem. Res.* 2026, 65, 9540–9562



Read Online

ACCESS |



Metrics & More



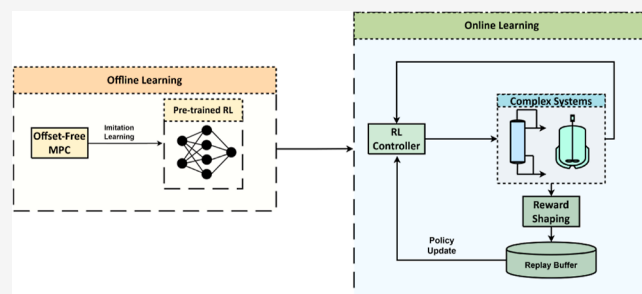
Article Recommendations



Supporting Information

ABSTRACT: Reinforcement learning (RL) is increasingly studied for process control, but online exploration can violate safety and operating constraints. Prior work from our research group reduces exploration by pretraining a twin delayed deep deterministic policy gradient (TD3) agent via behavioral cloning of an offset-free linear model predictive control (OF-MPC) policy. This approach significantly reduces unsafe online interaction and can exceed OF-MPC performance. However, our large-scale simulations show that MPC imitation alone, when paired with a one-step quadratic reward, can still lead to non-negligible steady-state error. To address this limitation, we modify the online phase in two ways.

First, we augment the MPC-style reward with an offset-aware shaping term that increases the learning signal near the set point. Second, we use a mixed-replay strategy that combines prioritized replay, a recency bias, and uniform sampling to balance adaptation and coverage under drift. The method is demonstrated on two systems: a multi-input multioutput (MIMO) styrene polymerization reactor and a large-scale ethylene-splitter distillation column simulated in Aspen Dynamics. In both nominal and disturbance scenarios, the proposed RL controller improves transient behavior and reduces steady-state error relative to the previous work.



1. INTRODUCTION

Process control is central to chemical engineering because it directly impacts process safety, economic performance, and

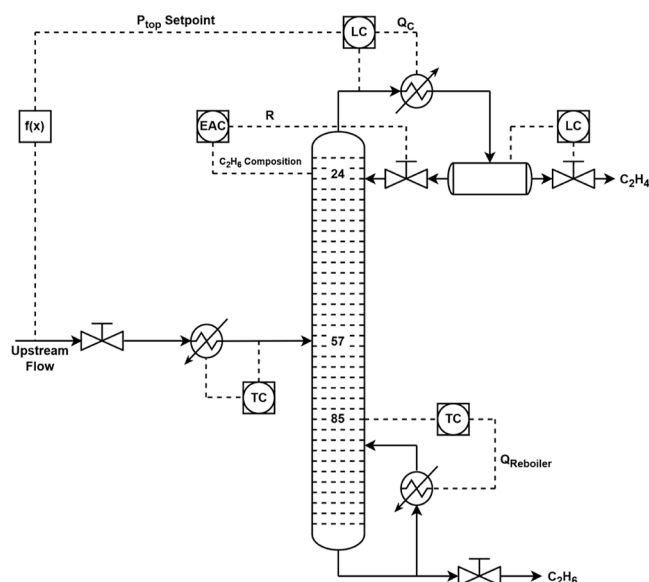


Figure 1. C_2 splitter process diagram.

product quality. In feedback control, manipulated variables are adjusted so that key outputs track desired set points despite disturbances and uncertainty.¹ Model Predictive Control (MPC) has been widely used in the process industries for decades because it can handle multivariable interactions and explicitly incorporate input and output constraints. At each sampling time, MPC uses a dynamic model to predict future system behavior over a finite horizon and solves an optimization problem to compute control actions.^{2,3}

The closed-loop performance of MPC depends heavily on the predictive model. Most industrial MPC applications rely on linear empirical models, often in step-response form, because they are easier to identify and maintain than first-principles models. Linear MPC is common in industrial practice; however, when the plant is nonlinear or operating under unseen disturbances, a linear model may yield inaccurate predictions and degrade tracking performance. In such cases, nonlinear MPC (NMPC), which uses a nonlinear predictive model, can be used to address system nonlinearities and process dynamics.^{4–10}

Received: January 19, 2026

Revised: April 1, 2026

Accepted: April 14, 2026

Published: April 30, 2026



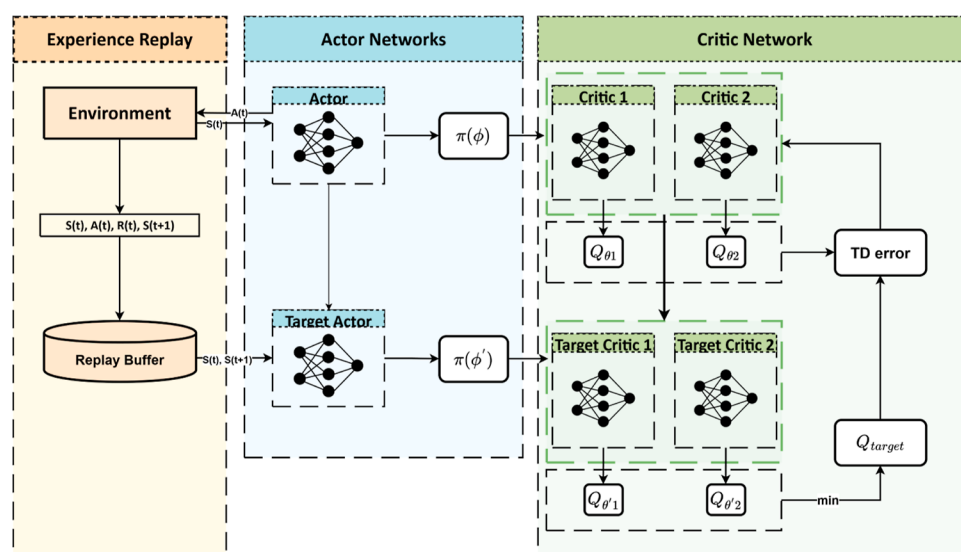


Figure 2. TD3 architecture and signal flow used in this work. The actor network maps the current state to the control action applied to the environment, and the resulting transition (s_t, a_t, r_t, s_{t+1}) is stored in the replay buffer. During training, sampled transitions are used to update the critic networks. The target actor computes the next action from the next state, and the two target critics evaluate this action–state pair. The minimum of the two target critic outputs is used to form the target value Q_{target} , which is then used in the temporal-difference (TD) error for critic learning.

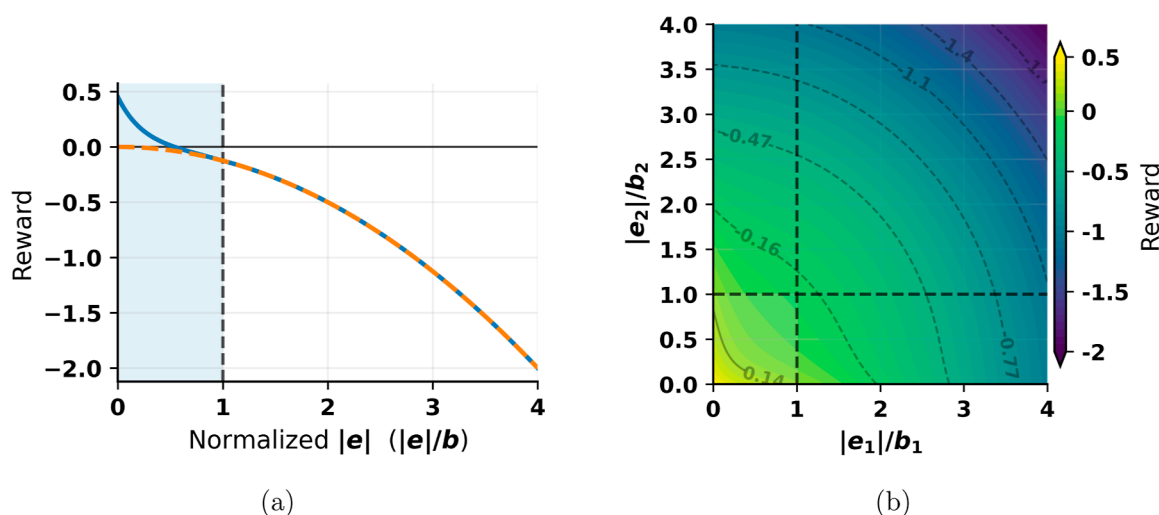


Figure 3. Visualization of the shaped reward in normalized error coordinates with $z_i = |e_i|/b_i$ and $\Delta u = 0$. In (a), the solid curve is the shaped reward and the dashed curve is the quadratic one-step tracking reward used for pretraining; the band edge is at $z = 1$ (vertical dashed line) and the shaded region indicates $z \leq 1$. In (b), the dashed lines indicate $z_1 = 1$ and $z_2 = 1$, and the shaded region corresponds to being inside the tracking band. Higher rewards are achieved closer to the set point ($z_i \rightarrow 0$), while the reward decays smoothly as tracking errors grow.

However, developing and maintaining nonlinear models can be resource-intensive, and solving NMPC online is typically more computationally demanding than linear MPC. As a result, linear MPC remains more common in industry, although its performance can deteriorate over time due to uncertainties, plant-model mismatch, and disturbances.

To address plant-model mismatch and unseen disturbances, closed-loop reidentification techniques can be used to update the model in closed loop.¹¹ The key challenge is that feedback control reduces the effective excitation available for identification because the controller stabilizes the system by rejecting disturbances.¹² This limitation, together with practical challenges in implementing and maintaining both nonlinear and linear MPC, motivates alternative approaches for improving closed-loop performance. More broadly, model development (first-principles, data-driven, or hybrid), robust and online

optimization algorithms, and control under uncertainty remain active topics in MPC research and industrial deployment.¹³

Reinforcement learning (RL) has been explored as a potential alternative because it is data-driven and can learn process dynamics through interaction with the system. In RL-based control, the agent selects actions to maximize a cumulative reward. Because the observed state directly influences the agent's actions, the RL problem is inherently a closed-loop, making it a strong fit for process control. In this setting, the agent determines control actions by considering disturbances and their future impacts.^{14–17} However, learning effective policies typically requires extensive interaction and extrapolation across many trajectories and operating scenarios. In practice, especially in chemical engineering processes, this level of exploration is often unacceptable because it can violate process safety and

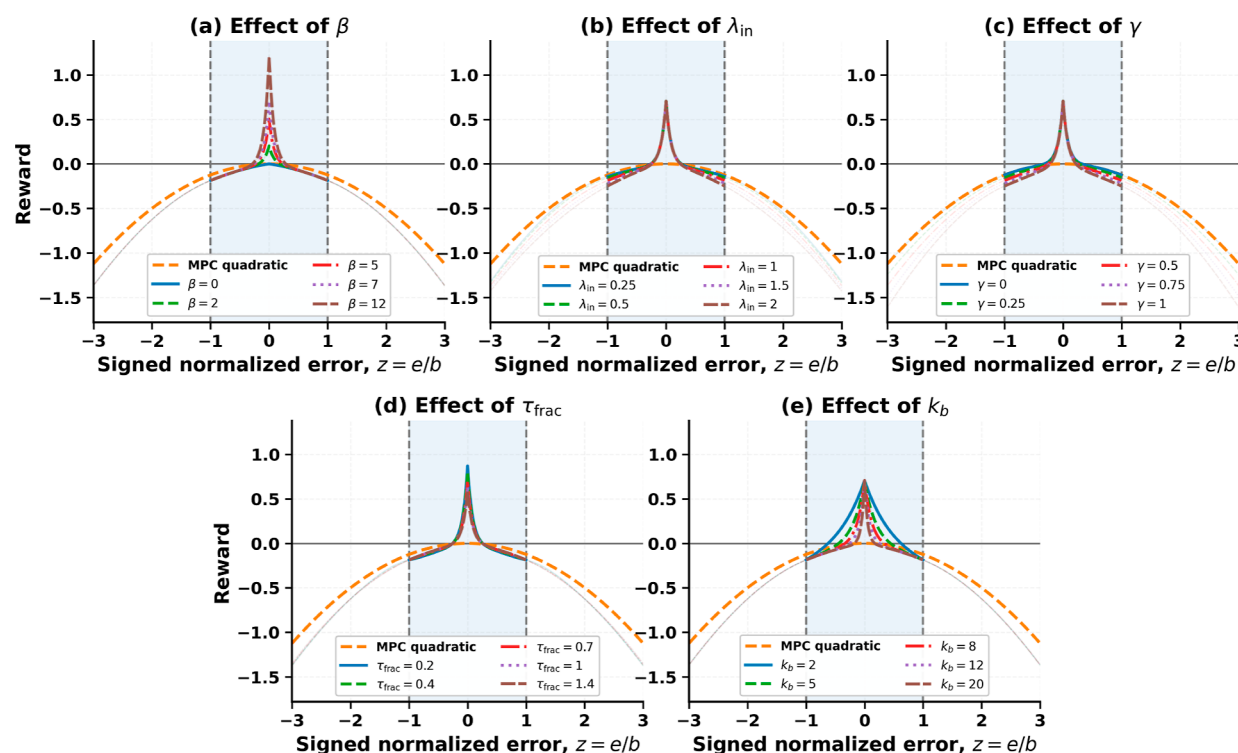


Figure 4. Qualitative effect of the main reward-shaping hyperparameters in signed normalized coordinates $z = e/b$ for the scalar case with $\Delta u = 0$. The dashed curve is the baseline one-step quadratic reward used during pretraining, and the shaded region indicates the in-band region $|z| \leq 1$. (a) Increasing β increases the magnitude of the in-band bonus. (b) λ_{in} controls the strength of the in-band quadratic penalty. (c) $\gamma = \gamma_{out} = \gamma_{in}$ controls the strength of the linear shaping terms near the band boundary. (d) τ_{frac} controls the smoothness of the transition near the band edge. (e) Increasing k_b concentrates the in-band bonus more strongly near the set point.

Table 1. Replay Hyperparameters Used in All Online Training Runs

parameter	meaning	value
φ_{per}	PER fraction of each minibatch	0.5
φ_{rec}	recent-window fraction of each minibatch	0.2
W	size of recent window $\mathcal{R}_t$	1000
α	priority exponent in (41)	0.6
ϵ	small constant for nonzero priorities	10^{-6}
β_0	initial IS exponent in (44)	0.4
β_{end}	final IS exponent	1.0
N_β	steps to anneal $\beta_0 \rightarrow \beta_{end}$	5×10^4
p_{min}, p_{max}	priority clipping bounds	$10^{-4}, 10^4$

operational constraints, making standard online RL difficult to apply directly.

Some studies seek to make RL more practical for process systems by reducing the burden of unrestricted online exploration. For example,¹⁸ incorporates prior control structure by embedding a PID layer within the policy, while¹⁹ uses principal component analysis and transfer learning to accelerate deep RL training in hydraulic fracturing. Other studies address safe deployment more explicitly by adding a supervisory layer during execution rather than relying on unrestricted online exploration. For example,²⁰ uses an RL policy as a fast surrogate for long-horizon MPC, and enforces closed-loop stability by accepting the RL action only when a Lyapunov-based decrease condition holds, otherwise switching to a stabilizing MPC-based backup. Similarly,²¹ proposes adaptive robust model predictive shielding, where candidate RL actions are checked using a predictive model and unsafe actions are replaced by an offline-

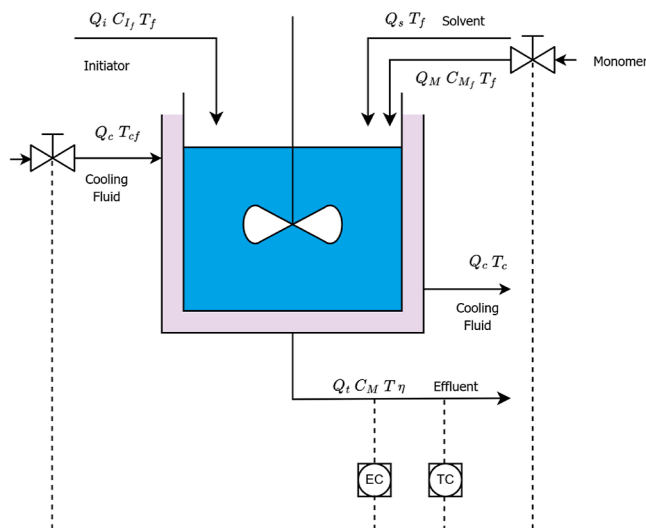


Figure 5. Process diagram of the styrene polymerization reactor.

Table 2. Parameter Settings for OF-MPC and the Online Shaped Reward (Polymer Reactor Case Study)

OF-MPC		modified reward function	
parameter	value	parameter	value
Q (diag)	(5, 1)	τ_{frac}	0.7
R (diag)	(1, 1)	$\gamma_{out} = \gamma_{in}$	0.5
prediction horizon N_p	9	β	7
control horizon N_c	3	k_b	12

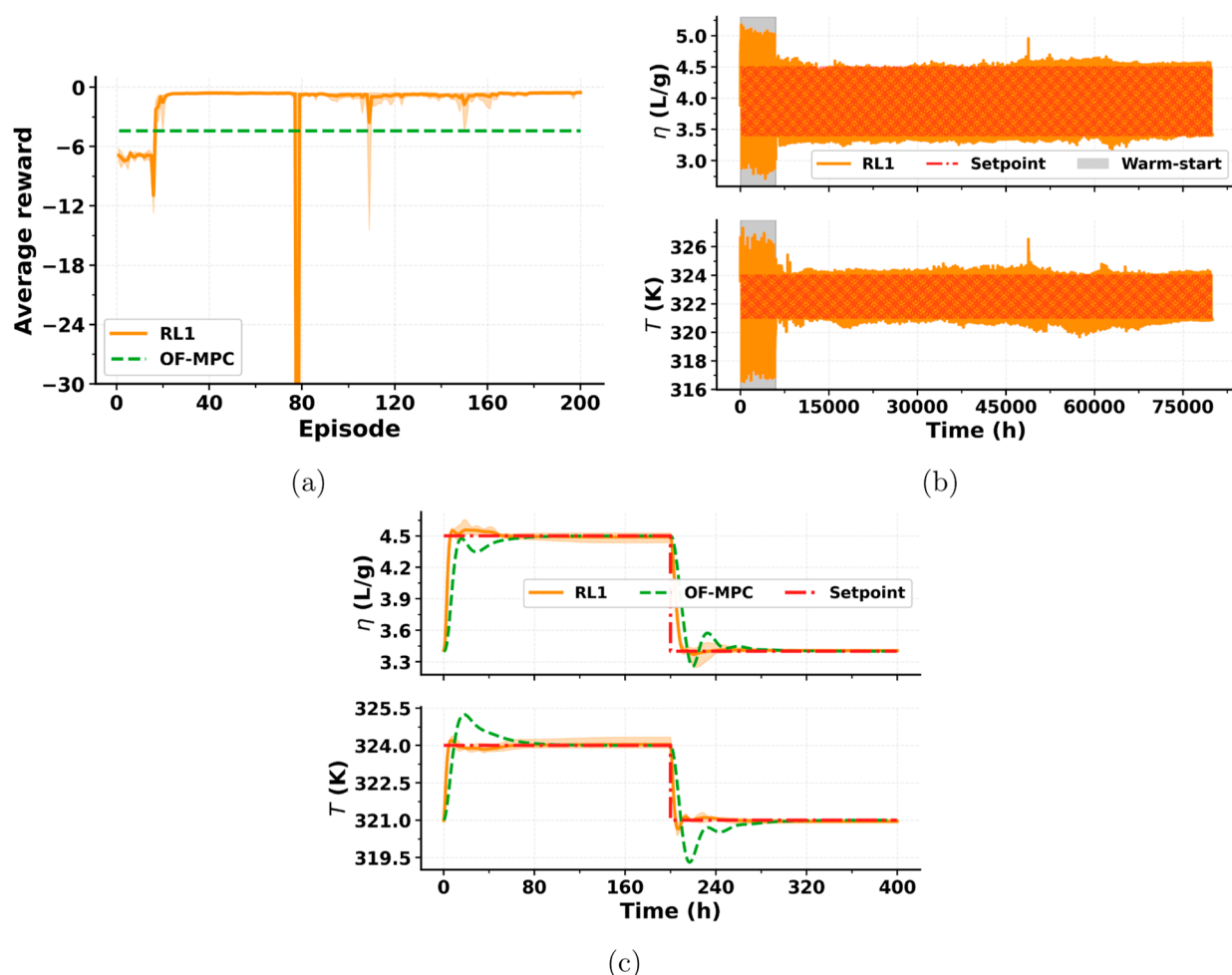


Figure 6. RL₁ vs OF-MPC under nominal conditions. (a) Episode-average reward during online training, averaged over five runs. The solid line shows the mean RL₁ reward and the shaded region shows the min-to-max range across runs. The dashed line indicates the OF-MPC baseline. (b) Full-horizon closed-loop trajectory ($\sim 8 \times 10^4$ h) from a representative run. The gray band marks the warm-start period. The dash-dot line denotes the set point sequence, and the solid line shows the RL₁ response. (c) Noise-free evaluation episode, shown as the mean and min-to-max range over five runs. RL₁ settles faster than OF-MPC, but a small steady-state offset remains. The dash-dot line denotes the set point sequence.

Table 3. Steady-State Absolute Tracking Error at the End of the Evaluation Episode, Reported as Mean (Std) over 5 Runs^a

	nominal		operational changes	
	SP(a)	SP(b)	SP(a)	SP(b)
		η [L/g]		
RL ₁	0.034 (0.024)	0.026 (0.020)	0.032 (0.020)	0.007 (0.006)
RL ₂	0.001 (0.001)	0.002 (0.003)	0.002 (0.003)	0.002 (0.002)
reduction (%)	98	90	92	76
		T [K]		
RL ₁	0.093 (0.085)	0.058 (0.043)	0.103 (0.133)	0.056 (0.038)
RL ₂	0.008 (0.009)	0.009 (0.005)	0.016 (0.012)	0.005 (0.005)
reduction (%)	92	84	84	90

^aReduction is computed as $100 \left(1 - \frac{RL_2}{RL_1} \right)$ using the mean values.

trained robust NMPC backup policy. These supervisory approaches can provide an additional safety layer, but they also require a predictive model, explicit constraints, and a backup stabilizing controller. In industrial practice, such prior process knowledge is typically available to some extent. However, for large-scale plants with uncertainty and plant-model mismatch, developing and maintaining a supervision layer with sufficient fidelity can still be demanding, and the resulting supervision may be conservative or add online

computation whenever the backup is invoked. Other studies instead reduce risky online exploration by training an RL agent on a high-fidelity process model (either a mathematical model or a simulator) before online deployment. Many RL approaches pretrain an RL agent using a surrogate model of the process.^{14,22,23} Recent work has also explored model-based policy optimization for feedback control of complex dynamic systems by using dynamic process models within the RL training loop, including modified MBPO formulations that reduce errors

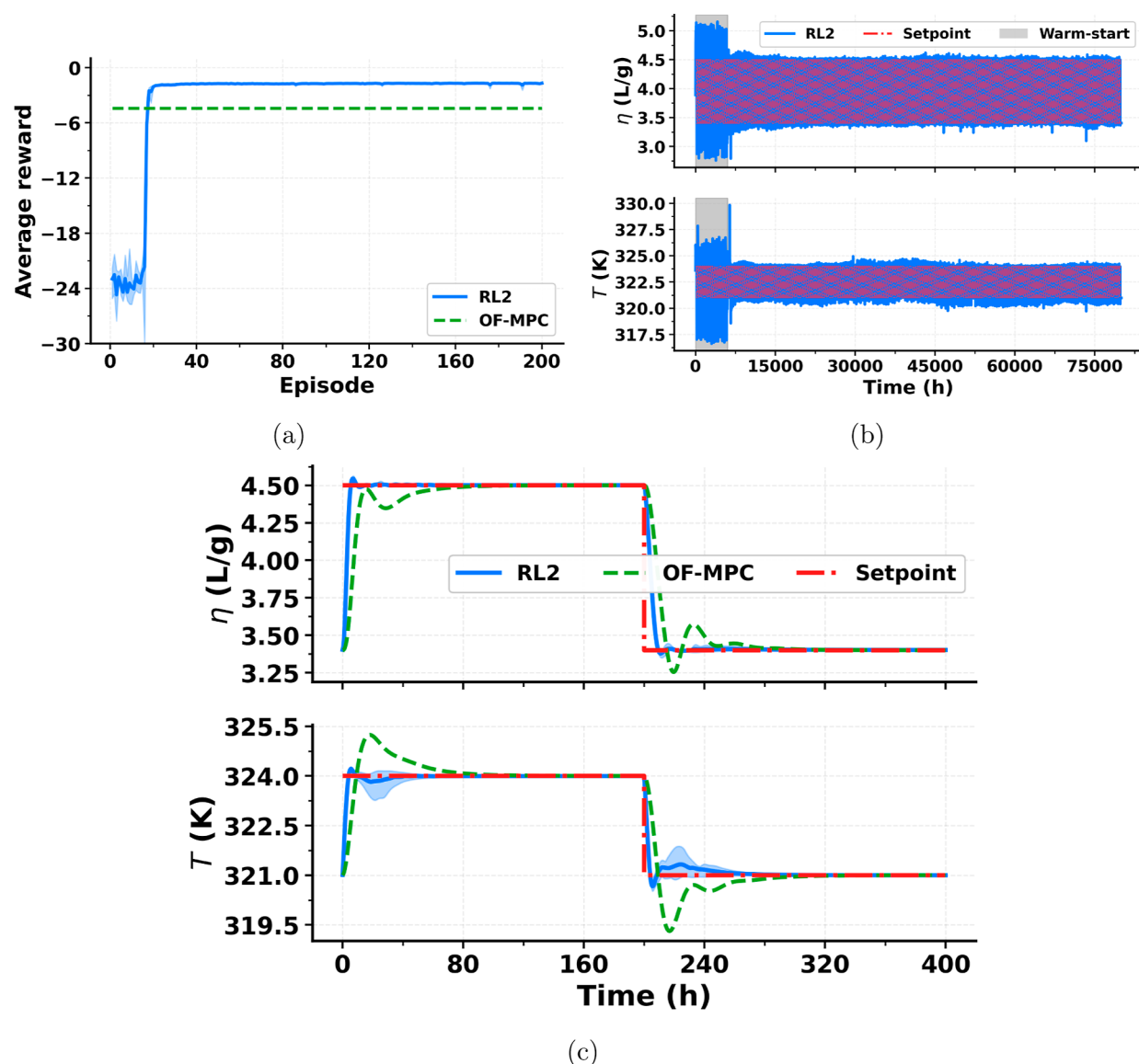


Figure 7. RL₂ vs OF-MPC under nominal conditions. (a) Episode-average reward during online training, averaged over five runs. The solid line shows the mean RL₂ reward, the shaded region shows the min-to-max range across runs, and the dashed line indicates the OF-MPC baseline. (b) Full-horizon closed-loop trajectory ($\sim 8 \times 10^4$ h) from a representative run. The gray band marks the warm-start period. The dash-dot line denotes the set point sequence. (c) Noise-free evaluation episode, shown as the mean and min-to-max range over five runs. RL₂ settles faster than OF-MPC and reduces the steady-state error relative to RL₁. The dash-dot line denotes the set point sequence.

associated with learned neural-network models.²⁴ However, developing a precise and accurate model presents the same challenges as MPC implementation. In other words, a surrogate model accurate enough for RL training is often also accurate enough to support an MPC design. This raises a practical question: if a reliable surrogate model is available, why not implement an MPC optimization problem given that MPC is more interpretable and widely adopted in industry? A practical alternative proposed recently is to pretrain an RL agent using a linear MPC formulation.²⁵ In that work, a TD3 agent is initialized from linear MPC trajectories and then deployed online. While this method reduces unsafe online exploration and works well for small- to medium-scale systems, its direct application to large-scale and industrial-scale systems is insufficient to achieve near offset-free performance. In such systems, stronger nonlinearities, tighter variable coupling, and large plant-model mismatch can limit the effectiveness of

pretraining (imitation learning or behavioral cloning) alone. This work targets that gap by introducing modifications that promote near offset-free performance in large-scale systems during online adaptation.

As discussed earlier, RL learns a control policy by maximizing a reward function, which is analogous to the objective function in optimal control theory.²⁶ For set point tracking problems, the reward must represent both fast transients and offset-free performance. Recent work has also shown that the choice of reward can strongly affect whether the learned policy actually prioritizes reaching and maintaining the target. In particular,²⁷ showed that classical dense, often quadratic, objectives can be misaligned with goal-reaching behavior, which further motivates revisiting reward design for tracking problems. In,²⁸ quadratic, absolute and square-root reward functions were analyzed for motion control. The results show that discounted quadratic returns can underemphasize small steady-state errors, leading to

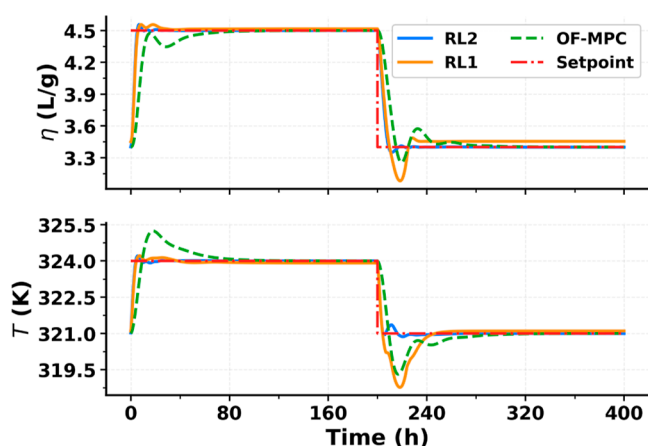


Figure 8. Last-episode tracking under nominal conditions for η and T in the evaluation episode with exploration noise disabled. Both RL_1 and RL_2 settle faster than OF-MPC, while RL_2 achieves the smallest steady-state error. The dash-dot line denotes the set point sequence.

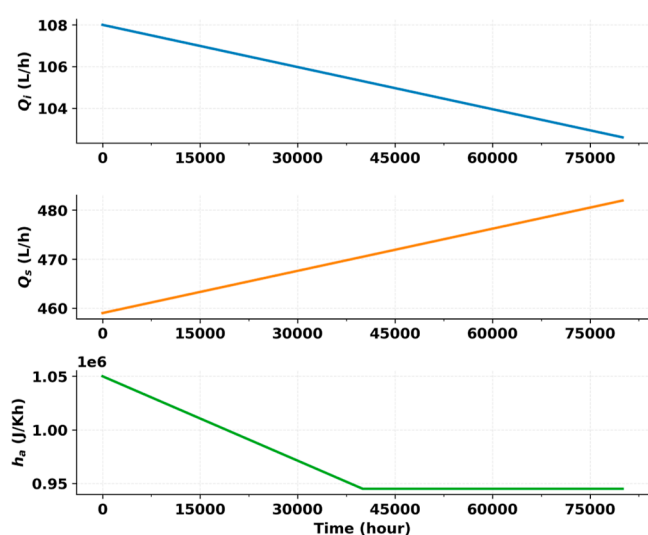


Figure 9. Slowly varying operating changes used in the long-run evaluation. The initiator flow Q_i decreases, the solvent flow Q_s increases, and the overall heat-transfer coefficient hA decreases and then saturates. These profiles represent gradual plant drift.

persistent offset even when transients improve. Since the one-step quadratic tracking reward used for MPC imitation matches an MPC stage cost, it may not provide enough learning signal to promote offset-free performance. In practice, offset-free MPC handles steady-state offsets by augmenting the model with disturbance states, but this structure is not directly enforced by the one-step imitation reward. One option is to augment the observation with the integral of the tracking error.^{29,30} However, augmenting the states changes the actor and critic inputs, which conflicts with pretraining and safe RL implementation in large-scale industrial systems. It also increases the observation dimension and does not always improve performance.³¹ Reward bonuses and related shaping ideas^{32,33} can help, but they are typically insufficient on their own.

Motivated by the above discussion and building on our previous work,^{12,25} this paper addresses a key limitation of MPC-pretrained RL in large-scale process systems. In our previous smaller-scale studies, the benefit of MPC-based pretraining relative to direct RL training was already established.

The focus here is therefore not to revisit TD3 training from scratch, but to address the next practical question: how to improve the online fine-tuning stage of MPC-pretrained RL so that it achieves more reliable near offset-free performance in large-scale industrial applications. While MPC imitation can work well for small- to medium-scale systems, it does not consistently achieve reliable performance in large-scale industrial applications. To close this gap, we extend the framework in two ways. First, we augment the MPC-style one-step tracking reward with an offset-aware shaping term that increases the learning signal near the set point. Second, we bias replay sampling using a mixed strategy that combines prioritized replay and a recency window, while retaining a uniform component for broad coverage. Section 2 introduces the large-scale distillation column case study and reviews OF-MPC and TD3 algorithm as baseline controllers. Section 3 presents the proposed modifications to the RL framework. To illustrate the approach on a nonlinear benchmark, a styrene polymerization reactor is used as an example, where the proposed method is compared to OF-MPC under both nominal and disturbance conditions. Section 4 evaluates the methodology on an industrial-scale distillation column in Aspen Dynamics, and Section 5 provides concluding remarks.

2. PRELIMINARIES

2.1. Motivating Example: Ethylene Splitter

Ethylene (C_2H_4) is one of the most widely produced petrochemicals in the industry. The C_2 splitter separates C_2H_4 from ethane (C_2H_6) and is typically the final purification step in ethylene production. The top product is ethylene, while the bottom stream, consisting mainly of ethane, is recycled to upstream units. This separation of C_2H_4 and C_2H_6 is among the most energy-intensive operations and requires a very large distillation column with more than 100 trays due to the small difference in relative volatility and high-pressure operating conditions.³⁴ As a result, optimization and control of the column are important to meet the desired C_2H_4 purity.

This column is challenging to control due to (1) strong coupling between top and bottom sections, (2) small differences in the relative volatility of C_2H_4 and C_2H_6 , (3) slow dynamics, (4) nonlinear process behavior, and (5) continuous changes in feed flow and composition.³⁵ In addition, high vapor and liquid traffic can exceed the hydraulic capacity and lead to flooding. Therefore, precise control of the C_2 splitter is critical.

Figure 1 shows the C_2 splitter process diagram. The required ethylene purity is typically above 99% to meet product specifications. To reduce energy consumption, the column is often operated close to the maximum allowable impurity level.³⁶ The column has two main proportional–integral (PI) controllers: a temperature controller on tray 85 that manipulates the reboiler duty, and a C_2H_6 composition controller on tray 24 that manipulates the reflux flow.

The large-scale distillation column is simulated in Aspen Dynamics to capture dynamic responses. Aspen Dynamics is interfaced with Python to create a working environment where the rigorous Aspen model computes the process outputs, while the RL and MPC computations are performed in Python. Step-test system identification is used to obtain the linear model required for MPC and RL implementation. More details about the column model and identification are provided in Supporting Information Section S1, Table S1, and Figure S1. The overall objective is to assess whether RL-based controllers can

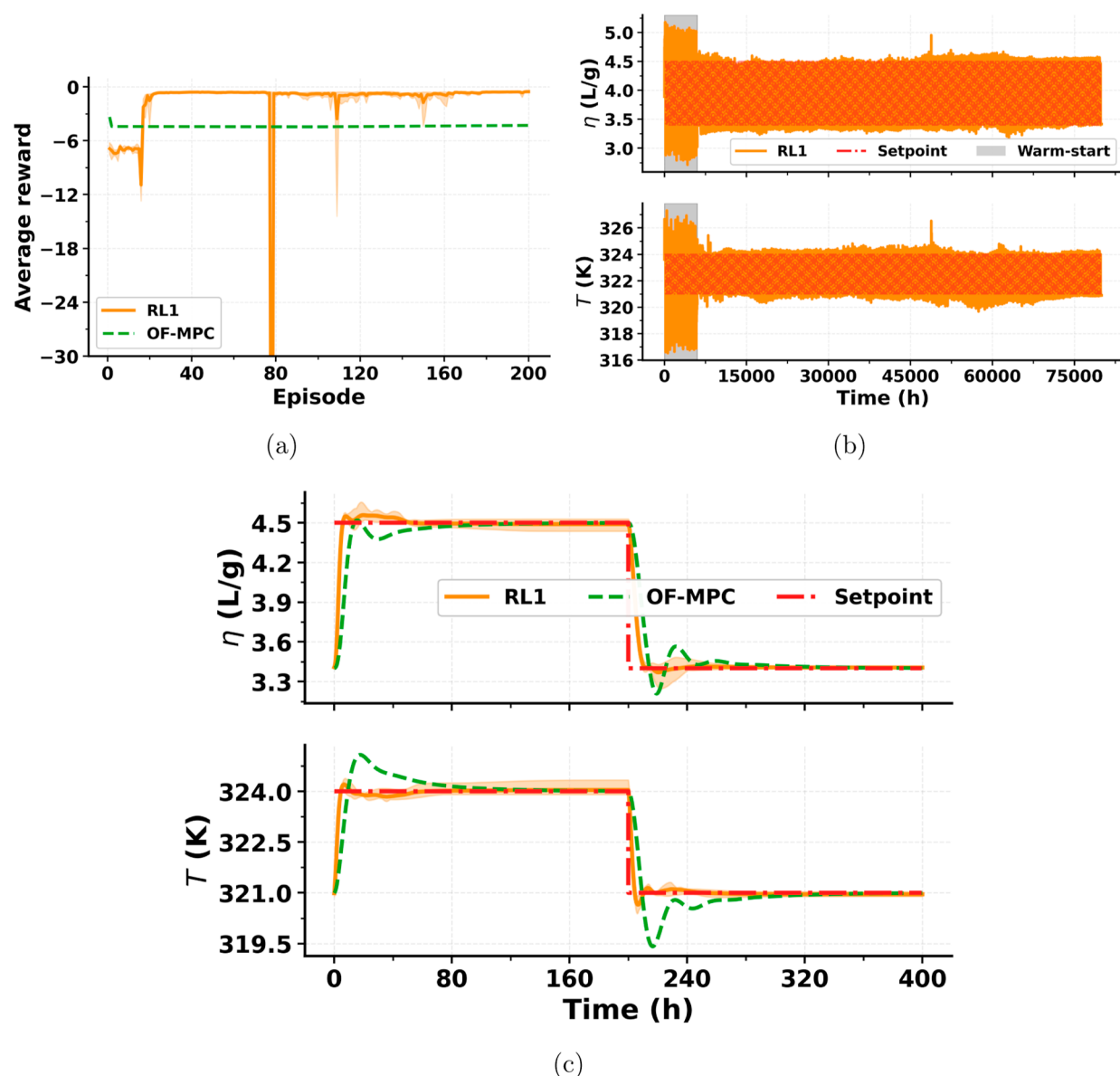


Figure 10. RL₁ vs OF-MPC under operational changes (Q_c decreases, Q_c increases, and hA decreases). (a) Episode-average reward during online training, shown as the mean over five runs. The solid line shows the mean RL₁ reward, the shaded region shows the min-to-max range across runs, and the dashed line indicates the OF-MPC baseline. (b) Full-horizon closed-loop trajectory ($\sim 8 \times 10^4$ h) from a representative run. The gray band marks the warm-start period. The dash-dot line denotes the set point sequence. (c) Last-episode tracking of η and T for the evaluation episode with exploration noise disabled. RL₁ settles faster than OF-MPC, but a small steady-state offset remains. The dash-dot line denotes the set point sequence.

outperform linear MPC for large-scale industrial systems, particularly under plant nonlinearities and disturbances.

2.2. Offset-Free Model Predictive Control

In this section, a brief review of OF-MPC is provided. OF-MPC addresses plant–model mismatch and unmeasured disturbances by augmenting the model with integrating disturbance states. These disturbance states are estimated online and allow the controller to remove steady-state offsets.^{37,38}

Consider the following discrete-time linear state-space model of the identified process

$$x_{t+1} = Ax_t + Bu_t \quad (1)$$

$$y_t = Cx_t + Du_t \quad (2)$$

where $x \in \mathbb{R}^{n_x}$, $u \in \mathbb{R}^{n_u}$, and $y \in \mathbb{R}^{n_y}$ are the state, manipulated input, and output vectors, respectively. $A \in \mathbb{R}^{n_x \times n_x}$, $B \in \mathbb{R}^{n_x \times n_u}$, $C \in \mathbb{R}^{n_y \times n_x}$, and $D \in \mathbb{R}^{n_y \times n_u}$ are the identified system matrices.

To achieve offset-free performance, the model is augmented with an integrating disturbance vector $d \in \mathbb{R}^{n_d}$

$$\begin{bmatrix} x_{t+1} \\ d_{t+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x_t \\ d_t \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_t \quad (3)$$

$$y_t = [C \ C_d] \begin{bmatrix} x_t \\ d_t \end{bmatrix} + Du_t \quad (4)$$

where B_d and C_d are tuning matrices with appropriate dimensions. The augmented model is then used within an offset-free MPC formulation.

At each sampling instant, given $\hat{x}_t$, $\hat{d}_t$, and the previously applied input u_{t-1} , the control action is obtained by solving the following finite-horizon quadratic program

$$\min_{u_t, \dots, u_{t+M-1}} \sum_{j=1}^P \|y_{t+j} - y_{SP,t+j}\|_Q^2 + \sum_{j=0}^{M-1} \|u_{t+j} - u_{t+j-1}\|_R^2 \quad (5)$$

$$\text{s.t. } x_{t+j+1} = Ax_{t+j} + B_d d_{t+j} + Bu_{t+j}, \quad j = 0, \dots, P-1 \quad (6)$$

$$d_{t+j+1} = d_{t+j}, \quad j = 0, \dots, P-1 \quad (7)$$

$$y_{t+j} = Cx_{t+j} + C_d d_{t+j} + Du_{t+j}, \quad j = 0, \dots, P-1 \quad (8)$$

$$u_{\min} \leq u_{t+j} \leq u_{\max}, \quad j = 0, \dots, M-1 \quad (9)$$

$$u_{t+j} = u_{t+M-1}, \quad j = M, \dots, P-1 \quad (10)$$

where $y_{SP,t+j}$ is the set point trajectory, P is the prediction horizon, M is the control horizon, $Q \geq 0$ and $R > 0$ are weighting matrices, and $\|x\|_Q^2 = x^T Q x$. The first control move u_t from the optimal sequence is applied to the plant, and the optimization is repeated at the next sampling instant.

Since the augmented states (x_p, d_t) are not directly measurable, a state estimator is required. In this work, a Luenberger observer is used

$$\begin{bmatrix} \hat{x}_{t+1} \\ \hat{d}_{t+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} \hat{x}_t \\ \hat{d}_t \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_t + L \left(y_t - [C \ C_d] \begin{bmatrix} \hat{x}_t \\ \hat{d}_t \end{bmatrix} - Du_t \right) \quad (11)$$

where L is the observer gain matrix.

OF-MPC serves as the benchmark controller in this study. This baseline allows us to evaluate whether the proposed RL controllers match or exceed OF-MPC performance in the case studies, particularly under plant nonlinearities and disturbances.

2.3. Reinforcement Learning

RL is a sequential decision-making framework in which an agent (controller) learns a policy by interacting with the environment (process).³⁹ An RL problem is often modeled as a Markov decision process (MDP), defined by the tuple $(S, \mathcal{A}, \mathcal{R}, p)$, where S is the state space, $\mathcal{A}$ is the action space, $\mathcal{R}$ is the reward function, and $p(s'|s, a)$ is the state transition probability. At each discrete time step t , the agent observes a state $s_t \in S$, selects an action $a_t \in \mathcal{A}$ according to a policy $\pi(a_t|s_t)$, and receives a scalar reward $r_t \in \mathbb{R}$. The environment then transitions to a new state s_{t+1} , and the agent aims to learn a policy π^* that maximizes the expected discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad J(\pi) = \mathbb{E}_{\pi}[G_t] \quad (12)$$

where $\gamma \in [0, 1)$ is the discount factor.

The state-value function under policy π is defined as

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t|s_t = s], \quad \forall s \in S \quad (13)$$

and the action-value function (Q-function) is

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t|s_t = s, a_t = a], \quad \forall s \in S, a \in \mathcal{A} \quad (14)$$

The optimal Q-function q^* satisfies the Bellman optimality equation

$$q^*(s, a) = \mathbb{E}[r_{t+1} + \gamma \max_{a'} q^*(s_{t+1}, a')|s_t = s, a_t = a] \quad (15)$$

Classical RL approaches include dynamic programming (DP), which assumes knowledge of $p(s'|s, a)$, and model-free methods such as Monte Carlo (MC) and temporal difference (TD) learning.³⁹ Q-learning is a widely used TD method, updating Q-values iteratively as

$$q(s_t, a_t) \leftarrow q(s_t, a_t) + \alpha[r_{t+1} + \gamma \max_{a'} q(s_{t+1}, a') - q(s_t, a_t)] \quad (16)$$

where α is the learning rate. For large-scale problems with continuous state or action spaces, function approximation (often via deep neural networks) is often used. This leads to deep RL algorithms such as the deep Q-network (DQN).

2.3.1. Actor-Critic and TD3. For continuous action spaces, actor-critic algorithms are widely used.^{40,41} The actor parametrizes the policy $\mu_{\theta}(s)$, and the critic estimates the action-value $Q_{\phi}(s, a)$. The deterministic policy gradient theorem⁴² gives

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim \mathcal{D}} [\nabla_a Q_{\phi}(s, a)|_{a=\mu_{\theta}(s)} \nabla_{\theta} \mu_{\theta}(s)] \quad (17)$$

In practice, the expectation is approximated using samples from the replay buffer $\mathcal{D}$.

Deep Deterministic Policy Gradient (DDPG) extends this idea using deep neural networks but it can suffer from training instability and Q-value overestimation.⁴³

As shown in Figure 2, the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm improves training stability by⁴⁴

1. Clipped double Q-learning: Two critics Q_{ϕ_1} and Q_{ϕ_2} are trained, and the smaller value is used in the target network.
2. Target policy smoothing: Target actions are perturbed with small, clipped noise to reduce sensitivity to sharp Q-function errors.
3. Delayed policy updates: The actor and target networks are updated less frequently than the critics.

The TD3 critic loss is

$$L(\phi_i) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(Q_{\phi_i}(s, a) - y)^2], \quad i \in \{1, 2\} \quad (18)$$

with target

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i}(s', \mu_{\theta'}(s') + \epsilon) \quad (19)$$

where $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$. Soft updates are used for the target networks

$$\phi'_i \leftarrow \tau \phi_i + (1 - \tau) \phi'_i, \quad \theta' \leftarrow \tau \theta + (1 - \tau) \theta' \quad (20)$$

The target action is also projected onto the admissible input set, that is

$$a' = \Pi_{\mathcal{U}}(\mu_{\theta'}(s') + \epsilon) \quad (21)$$

and a' is used inside $Q_{\phi_i}(s', a')$.

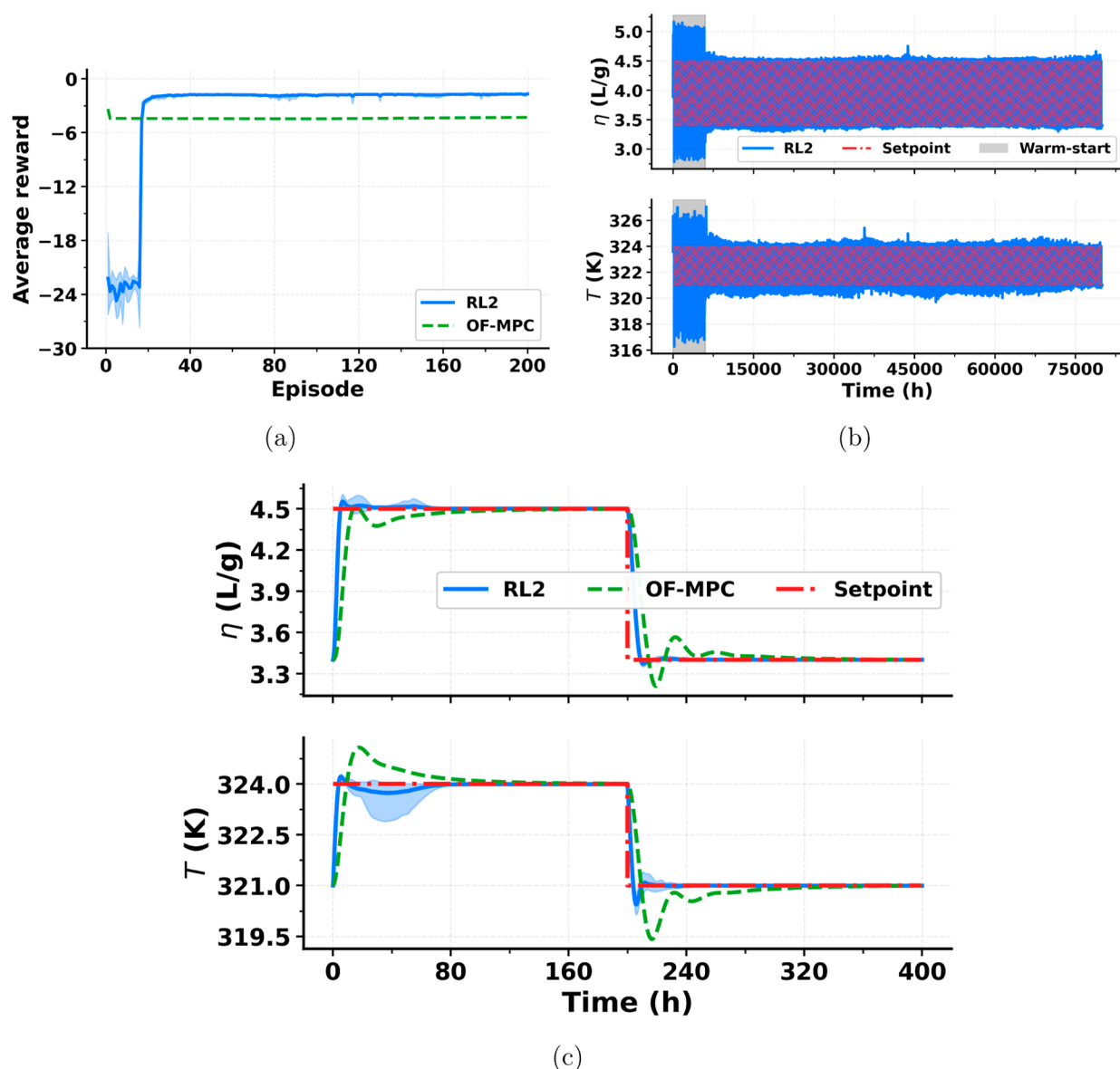


Figure 11. RL₂ vs OF-MPC under operational changes (Q_c decreases, Q_c increases, and hA decreases). (a) Episode-average reward during online training, shown as the mean over five runs. The solid line shows the mean RL₂ reward, the shaded region shows the min-to-max range across runs, and the dashed line indicates the OF-MPC baseline. (b) Full-horizon closed-loop trajectory ($\sim 8 \times 10^4$ h) from a representative run. The gray band marks the warm-start period. The dash-dot line denotes the set point sequence. (c) Last-episode tracking of η and T for the evaluation episode with exploration noise disabled. RL₂ settles faster than OF-MPC and reduces the steady-state offset relative to RL₁. The dash-dot line denotes the set point sequence.

In this work, TD3 is used as the baseline RL method because it reduces Q -value overestimation compared to DDPG and is widely used in continuous-control tasks. This makes TD3 a suitable benchmark for large-scale process control case studies, where stability and precise tracking are important and exploration must be limited.

While TD3 is used here as the baseline actor-critic method because of its strong performance and improved stability in continuous-control problems, the proposed online modifications are not tied entirely to TD3. In particular, the reward-shaping design is general and could in principle be used with other actor-critic or policy-gradient methods. The mixed-replay modification, however, is most naturally suited to off-policy algorithms that learn from a replay buffer, such as TD3, DDPG, or Soft Actor-Critic (SAC). In contrast, an on-policy method

such as Proximal Policy Optimization (PPO) could use the shaped reward, but not the replay modification in the same form.

3. PROPOSED METHODOLOGY

The proposed methodology consists of two phases: (i) a pretraining phase and (ii) an online deployment phase. In the pretraining phase, the TD3 agent is initialized by behavioral cloning to imitate the OF-MPC policy, using trajectories generated from the augmented linear state-space model and the corresponding OF-MPC optimization solutions. This initialization provides an MPC-like policy that can reduce the need for risky exploration during online deployment. In the online phase, the agent is fine-tuned using an offset-aware shaped reward and a mixed-sampling replay buffer. These modifications encourage offset-free behavior while retaining

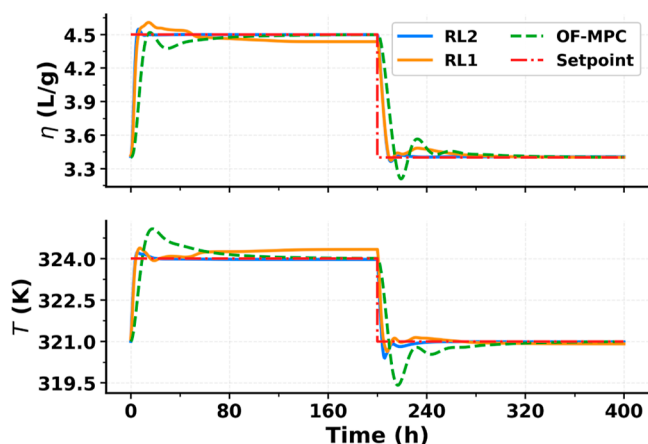


Figure 12. Last-episode tracking under operational changes for η and T in the evaluation episode with exploration noise disabled. Both RL_1 and RL_2 settle faster than OF-MPC, and RL_2 achieves the smallest steady-state error. The dash-dot line denotes the set point sequence.

the benefits of MPC pretraining. In large-scale nonlinear systems, training with only the quadratic one-step tracking reward can be insufficient to remove steady-state offset. The proposed online modifications address this limitation and support stable online adaptation.

3.1. Pretraining (MPC Imitation)

The pretraining phase constructs an offline data set from the OF-MPC described in Section 2.2 and uses it to initialize a TD3 agent through behavioral cloning.

3.1.1. Model Used for Pretraining. Offline data are generated using the augmented model in Section 2.2 and eqs 3 and 4. This is the same identified linear model used for the OF-MPC implementation (see Supporting Information Section S5), and no additional modeling is introduced here. We use the constant output disturbance model with $B_d = 0$ and $C_d = I$. The augmented state is defined as $\tilde{x}_t = [x_t^T, d_t^T]^T$. For simplicity, we denote the augmented matrices in 3 and 4 as $\tilde{A}$, $\tilde{B}$, $\tilde{C}$. These matrices are used during pretraining to generate one-step transitions under the first control move computed by OF-MPC.

3.1.2. Offline Data set Generation. For each sample, we (i) draw an augmented state $\tilde{x} \in \mathcal{X}$ with predefined bounds, (ii) draw a set point $y_{sp} \in \mathcal{Y}$, and (iii) draw a feasible previous input $u^- \in U$ (all in deviation coordinates). We then solve the OF-MPC problem of Section 2.2 with horizons (N_p , N_c) to

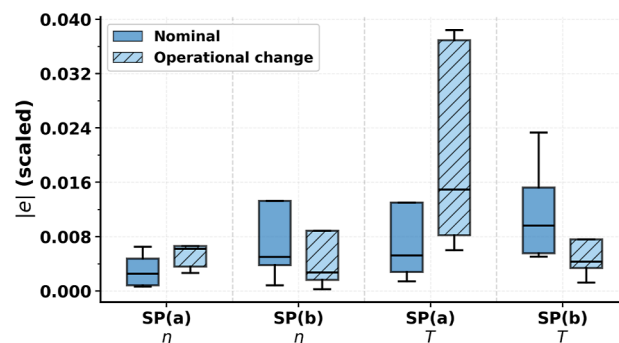


Figure 14. Box plots of the absolute scaled steady-state tracking error $|e|$ at the end of the evaluation episode for RL_2 , computed over five runs, under nominal and operational-change conditions. For each output and set point change, the final error remains low in both cases and is of the same overall order of magnitude. The operational-change case does not show a systematic improvement over the nominal case. Instead, some groups are similar or slightly smaller, while others, particularly T at SP(a), show a larger spread under operational changes.

obtain the optimal sequence $U^* = \{u_0^*, \dots, u_{N_c-1}^*\}$ and use the first move $u^{MPC} = u_0^*$. Using eqs 3 and 4, we propagate one step as

$$\tilde{x}_+ = \tilde{A}\tilde{x} + \tilde{B}u^{MPC}, \quad y_+ = \tilde{C}\tilde{x}_+ + Du^{MPC}$$

We record the transition

$$s = \begin{bmatrix} \tilde{x} \\ y_{sp} \\ u^- \end{bmatrix}, \quad a = u^{MPC}, \quad s' = \begin{bmatrix} \tilde{x}_+ \\ y_{sp} \\ u^{MPC} \end{bmatrix}$$

together with the pretraining reward in 22.

3.1.3. Offline Reward Used for Pretraining. For each transition, the reward is the one-step quadratic tracking reward evaluated under the OF-MPC first move

$$r = -(\|y^+ - y_{sp}\|_Q^2 + \|\Delta u\|_R^2). \quad (22)$$

where

$$\tilde{x}^+ = \tilde{A}\tilde{x} + \tilde{B}u^{MPC}, \quad y^+ = \tilde{C}\tilde{x}^+ + Du^{MPC}, \\ \Delta u = u^{MPC} - u^-$$

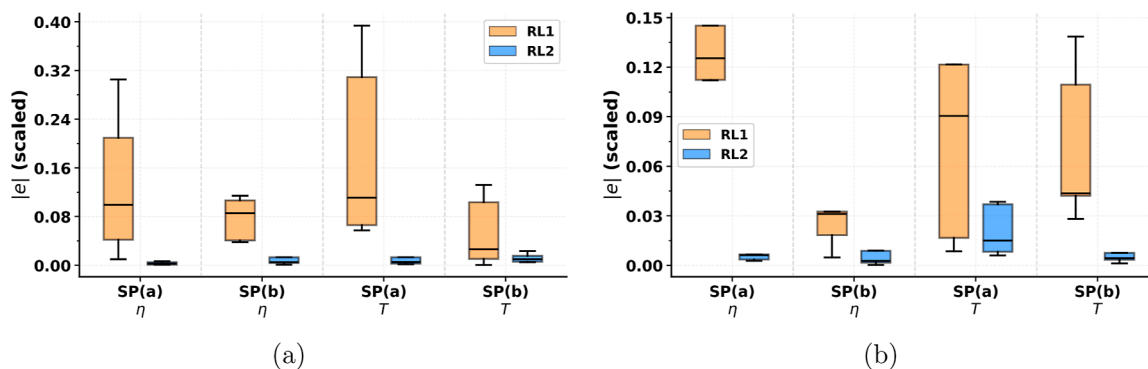


Figure 13. Box plots of the absolute scaled steady-state tracking error $|e|$ at the end of the evaluation episode, computed over five runs. (a) Nominal conditions. (b) Operational changes. For each output and set point change, RL_2 shows a lower median error and generally smaller variability than RL_1 . The same y -axis scale is used in both panels for direct comparison.

Here, $Q \geq 0$ and $R > 0$ are the same penalty matrices as in the OF-MPC objective.

3.1.4. Behavioral Cloning of OF-MPC. Let $\mathcal{D} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^N$ be the generated data set with

$$s_i = \begin{bmatrix} \tilde{x}_i \\ y_{SP,i} \\ u_i^- \end{bmatrix}, a_i = u_i^{\text{MPC}}, \text{ and } r_i \text{ as in 22. The actor } \mu_\theta \text{ is initialized}$$

by supervised imitation of the OF-MPC first move under input bounds $\mathcal{U} = \{u | u_{\min} \leq u \leq u_{\max}\}$

$$\theta_{\text{BC}} \in \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \|\Pi_{\mathcal{U}}(\mu_\theta(s_i)) - a_i\|_2^2 \quad (23)$$

where $\Pi_{\mathcal{U}}$ denotes the Euclidean projection onto $\mathcal{U}$. This yields a policy that approximates the OF-MPC first move while remaining within actuator limits. Equation 23 corresponds to behavioral cloning, that is, supervised imitation learning in which the actor is trained to match the OF-MPC first move over the offline data set. Its role here is not to introduce a loss

different from standard supervised regression, but to initialize the same bounded actor used later for TD3 fine-tuning, thereby providing a consistent transition from offline OF-MPC imitation to online actor-critic adaptation.

3.1.5. Offline TD3 Critic Training. Twin critics Q_{ϕ_1} and Q_{ϕ_2} are trained on $\mathcal{D}$ using TD3 targets consistent with 22

$$t_i = r_i + \gamma \min_{j=1,2} Q_{\phi_j}(s'_i, \Pi_{\mathcal{U}}(\mu_{\theta'}(s'_i) + \varepsilon)),$$

$$\varepsilon \sim \text{clip}(\mathcal{N}(0, \sigma^2 I), -c, c) \quad (24)$$

$$L_Q(\phi_j) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (Q_{\phi_j}(s_i, a_i) - t_i)^2, \quad j \in \{1, 2\} \quad (25)$$

Actor updates during pretraining minimize only the cloning objective 23, and the target networks are updated softly with rate τ . The algorithm for this pretraining procedure is provided in Algorithm 1.

Algorithm 1: Pretraining via OF-MPC imitation and offline TD3

Input: Augmented model of Eqs. (3)–(4), input bounds $\mathcal{U}$, OF-MPC with horizons

(N_P, N_C) , capacity N

1 **Data generation:**

2 **for** $i = 1$ **to** N **do**

3 Sample $\tilde{x}_i \in \mathcal{X}$, $y_{SP,i} \in \mathcal{Y}$, and $u_i^- \in \mathcal{U}$

4 Solve OF-MPC to obtain u_i^{MPC}

5 Propagate one step with Eqs. (3)–(4) to get $(\tilde{x}_{i+}, y_{i+})$

6 Form $s_i = \begin{bmatrix} \tilde{x}_i \\ y_{SP,i} \\ u_i^- \end{bmatrix}$, $a_i = u_i^{\text{MPC}}$, $s'_i = \begin{bmatrix} \tilde{x}_{i+} \\ y_{SP,i} \\ u_i^{\text{MPC}} \end{bmatrix}$, and r_i by (22) and store (s_i, a_i, r_i, s'_i) in $\mathcal{D}$

7 **end**

8 **Offline training:**

9 **for** $epoch = 1, 2, \dots$ **do**

10 Sample minibatch $\mathcal{B} \subset \mathcal{D}$

11 **Critics:** update Q_{ϕ_1} and Q_{ϕ_2} by minimizing $L_Q(\phi_j)$

12 **Actor (cloning):** update μ_θ by minimizing (23) with projection $\Pi_{\mathcal{U}}$

13 Soft-update targets with rate τ

14 **end**

3.1.5.1. Sampling Sets and Implementation Details. The offline data set $\mathcal{D}$ is generated by sampling $(\tilde{x}, y_{\text{SP}}, u^-)$ from predefined sets $(\mathcal{X}, \mathcal{Y}, \mathcal{U})$. The augmented state limits in $\mathcal{X}$ are selected using the minimum and maximum component values of the identified states x_i and the disturbance states d_i on the data used for model identification. The set point set $\mathcal{Y}$ is chosen to cover the normal operating range used in closed-loop tests. The input set $\mathcal{U}$ is defined by the actuator bounds $u_{\min} \leq u \leq u_{\max}$ and all sampled values are expressed in deviation coordinates.

For the case studies in this work, the offline pretraining data set contains about 3×10^6 transitions. The actor and critic networks use five hidden layers with 512 neurons per layer. All

offline samples are generated using the augmented linear state-space model in Section 2.2. Pretraining is performed on an NVIDIA RTX 4090 GPU and takes about 12 h.

For clarity, the offline pretraining data set is generated directly from the identified augmented linear model by sampling $\tilde{x}$, y_{SP} , and u^- , solving OF-MPC, and propagating the system one step using eqs 3 and 4. Thus, this stage does not use an online observer driven by plant measurements. The Luenberger observer in eq 11 is used only in the online OF-MPC implementation to estimate the augmented states during closed-loop operation. A stochastic estimator such as a Kalman filter could also be used in principle, but this would mainly affect

the online estimation layer rather than the basic construction of the offline behavioral-cloning data set.

3.2. Online Fine-Tuning: Motivation and Design Choices

While the one-step quadratic tracking reward used for pretraining (eq 22) is effective for initializing the policy, continuing with this reward can lead to a small but persistent steady-state error,^{28,29,45} especially in large-scale and nonlinear plants. Three factors contribute to this behavior. The first factor is that when $\gamma < 1$, the discounted return places less weight on rewards far in the future. In practice, this can bias learning toward reducing transient errors rather than removing small steady-state errors.²⁸ Second, the quadratic move penalty discourages the small corrective actions, which can slow final removal of offset. Finally, the replay buffer is dominated by transitions from earlier policies and transient responses, so uniform sampling can undersample the regimes that matter for fine corrections near the set point and can slow adaptation under drift.

To address these issues, the online phase augments the baseline with two modifications: (i) offset-aware reward shaping that strengthens the learning signal near the set point, and (ii) a mixed-sampling replay strategy that combines prioritized experience replay with a mild recency bias.

Remark 1 across the case studies discussed in this work, and consistently across repeated runs, we observed that using the MPC-style one-step quadratic reward alone during online fine-tuning can still leave a small residual steady-state offset, especially in larger-scale or more nonlinear systems. This reward remains useful because it is consistent with the OF-MPC stage cost and provides a stable warm-start for policy initialization. However, during online fine-tuning, it may provide only a weak learning signal for removing small residual errors near the set point. In particular, discounting places greater emphasis on transient improvement than on small long-term offsets, while the move penalty can discourage the small corrective actions needed close to the target. As a result, the policy may learn stabilizing and reasonably accurate tracking behavior, yet still settle with a small offset. This practical observation motivates retaining the MPC-style reward as a warm-start and then introducing the proposed shaped reward during online adaptation to place additional emphasis on near-set point correction and improve offset removal.

3.2.1. Reward Shaping with Relative Tracking Bands.

The online reward is built from a one-step quadratic tracking and move penalty that matches the stage cost used in the offset-free MPC formulation. We then add band-based shaping to explicitly emphasize near-set point behavior and reduce steady-state offset.

3.2.1.1. Scaled Error and Input Move. All terms are computed in the same scaled deviation coordinates used by the agent. Let

$$e(k) = y(k) - y_{sp}(k), \quad \Delta u(k) = u(k) - u(k-1) \quad (26)$$

and let $e_s(k)$ and $\Delta u_s(k)$ denote the corresponding scaled vectors used by the agent (obtained from the min–max scaling used in training).

3.2.1.2. Relative Tracking Band. A per-output tolerance band is defined in physical units as

$$b_i^{\text{phys}}(k) = \max(k_{\text{rel},i}|y_{sp,i}(k)|, b_i^{\text{floor,phys}}) \quad (27)$$

where $k_{\text{rel},i}$ sets a relative tolerance and $b_i^{\text{floor,phys}}$ prevents unrealistically small bands near zero set points. The band is converted to scaled coordinates using the output ranges

$$\Delta y_i = y_i^{\text{max}} - y_i^{\text{min}}, \quad b_i(k) = \frac{b_i^{\text{phys}}(k)}{\Delta y_i} \quad (28)$$

3.2.1.3. Smooth In-Band Gate. To avoid a hard switching behavior at the band boundary, we define a smooth per-output indicator

$$s_i(k) = \sigma\left(\frac{b_i(k) - |e_{s,i}(k)|}{\tau_i(k)}\right), \quad \tau_i(k) = \tau_{\text{frac}} b_i(k) \quad (29)$$

where $\sigma(\cdot)$ is the logistic function and $\tau_{\text{frac}} > 0$ controls the transition width. A single scalar in-band weight is then computed using a geometric mean gate

$$w_{\text{in}}(k) = \left(\prod_{i=1}^{n_y} s_i(k)\right)^{1/n_y} \quad (30)$$

so that $w_{\text{in}}(k)$ approaches 1 only when all outputs are within their bands and approaches 0 when one or more outputs are outside.

3.2.1.4. One-Step Quadratic Stage Cost. The baseline one-step quadratic penalty is

$$J_{\text{quad}}(k) = \sum_{i=1}^{n_y} q_i e_{s,i}(k)^2 + \sum_{j=1}^{n_u} r_j \Delta u_{s,j}(k)^2 \quad (31)$$

where $Q = \text{diag}(q_i)$ and $R = \text{diag}(r_j)$.

3.2.1.5. Band-Shaped Reward. Let $J_e(k) = \sum q_i e_{s,i}(k)^2$ and $J_{\Delta u}(k) = \sum r_j \Delta u_{s,j}(k)^2$. We optionally reweight the tracking term inside the band using

$$J_e^{\text{eff}}(k) = (1 - w_{\text{in}}(k))J_e(k) + w_{\text{in}}(k)\lambda_{\text{in}}J_e(k) \quad (32)$$

where λ_{in} is a tuning parameter.

We also include optional linear terms to shape behavior near the band edge. Define the slope at the band boundary as

$$m_i(k) = 2q_i b_i(k) \quad (33)$$

and the outside-band overflow as $\delta_i^{\text{out}}(k) = \max(|e_{s,i}(k)| - b_i(k), 0)$. The outside and inside linear penalties are

$$J_{\text{lin}}^{\text{out}}(k) = (1 - w_{\text{in}}(k)) \sum_i \gamma_{\text{out}} m_i(k) \delta_i^{\text{out}}(k) \quad (34)$$

$$J_{\text{lin}}^{\text{in}}(k) = w_{\text{in}}(k) \sum_i \gamma_{\text{in}} m_i(k) \min(|e_{s,i}(k)|, b_i(k)) \quad (35)$$

Finally, to explicitly reward eliminating small residual errors, we add an in-band bonus based on the normalized error

$$z_i(k) = \frac{|e_{s,i}(k)|}{b_i(k)} \quad (36)$$

and a shaping function $\phi(z) \in [0, 1]$. In this work we use an exponential form

$$\phi(z) = \frac{e^{-k_b z} - e^{-k_b}}{1 - e^{-k_b}}, \quad z \in [0, 1] \quad (37)$$

with clipping of z to $[0, 1]$ in implementation. The bonus is

$$J_{\text{bonus}}(k) = w_{\text{in}}(k) \beta \sum_{i=1}^{n_y} q_i b_i(k)^2 \phi(z_i(k)) \quad (38)$$

where β controls its magnitude.

The final shaped reward is

$$r(k) = -(J_e^{\text{eff}}(k) + J_{\Delta u}(k) + J_{\text{lin}}^{\text{out}}(k) + J_{\text{lin}}^{\text{in}}(k)) + J_{\text{bonus}}(k) \quad (39)$$

This design reduces sensitivity to operating point magnitude through the relative band definition (27) and increases pressure to remove small steady errors through the in-band bonus (38).

3.2.1.6. Visualization of the Shaped Reward. Figure 3 illustrates the proposed shaped reward as a function of the normalized tracking error. We define the normalized error as

$$z_i = \frac{|e_i|}{b_i} \quad (40)$$

where b_i is the (set point-dependent) tracking band for output i and the band edge corresponds to $z_i = 1$. Figure 3a shows the scalar reward $r(z)$ for a single-output case with $\Delta u = 0$, comparing the shaped reward (solid) against the quadratic one-step tracking cost used for pretraining (dashed). Inside the band ($z \leq 1$), the smooth gate activates an additional bonus that increases as $z \rightarrow 0$, encouraging the controller to keep reducing small residual errors rather than settling with a steady-state offset. Outside the band ($z > 1$), the gate deactivates and the reward is dominated by the quadratic penalty, yielding a smooth decay with increasing error magnitude. Figure 3b shows the corresponding two-output reward surface over (z_1, z_2) , highlighting the region where both outputs are within their respective bands.

Remark 2 The hyperparameters used to generate Figure 3 are chosen for visualization clarity and do not necessarily match the exact values used in all training runs.

3.2.1.7. Interpretation of Reward-Shaping Hyperparameters. To further illustrate the role of the main shaping parameters, Figure 4 shows the scalar reward in signed normalized coordinates

$$z = \frac{e}{b}$$

for $\Delta u = 0$, where the shaded region corresponds to $|z| \leq 1$. In all panels, the dashed curve denotes the baseline one-step quadratic reward used during pretraining, and the colored curves denote the shaped reward for different choices of one hyperparameter while the others are held fixed.

Figure 4a shows the effect of β , which scales the in-band bonus term $J_{\text{bonus}}(k)$ in eq 38. Increasing β raises the reward more strongly near $z = 0$, thereby increasing the incentive to remove small residual errors rather than settling with a small steady-state offset. If β is too small, the in-band shaping becomes weak and the reward approaches the baseline quadratic form. If it is too large, the reward becomes overly peaked near the set point and can overemphasize very small corrections.

Figure 4b shows the effect of λ_{in} , which scales the in-band quadratic term in eq 32. Smaller values reduce the quadratic penalty more strongly once the outputs are inside the tracking band, while larger values keep the shaped reward closer to the baseline MPC-style objective. Thus, λ_{in} controls how strongly the reward departs from the original quadratic tracking penalty in the near-set point region.

Figure 4c shows the effect of the common linear-shaping value $\gamma = \gamma_{\text{out}} = \gamma_{\text{in}}$. Larger γ increases the contribution of the linear shaping terms in eqs 34 and 35, making the reward change more strongly with distance from the band boundary. In the implementation used here, γ_{out} and γ_{in} were chosen equal so that the near-band shaping remains symmetric across the inside and outside regions.

Figure 4d shows the effect of τ_{frac} which determines the transition width in the smooth gate of eq 30. Smaller values make the transition at the band boundary sharper, while larger values spread the transition over a wider region. A very small τ_{frac} makes the reward closer to a hard band switch, whereas a larger value yields a smoother but less localized shaping effect near $|z| = 1$.

Figure 4e shows the effect of k_b , which controls the decay rate of the exponential shaping function $\phi(z)$ in eq 37. Larger k_b concentrates the bonus more strongly near $z = 0$, whereas smaller k_b spreads the bonus more broadly across the in-band region. In this sense, β primarily controls the overall bonus magnitude, while k_b controls how locally that bonus is concentrated around the set point.

In the case studies of this work, the shaping parameters were selected heuristically to preserve a smooth reward profile while still providing a visible increase in learning pressure near the set point. In particular, $\lambda_{\text{in}} = 1$ was retained so that the in-band quadratic term remains consistent with the baseline objective, $\gamma_{\text{out}} = \gamma_{\text{in}} = 0.5$ was used to keep the linear shaping terms moderate and symmetric, $\tau_{\text{frac}} = 0.7$ was chosen to avoid a sharp transition at the band boundary, and $\beta = 7$ with $k_b = 12$ was chosen to provide a noticeable but localized in-band bonus without overwhelming the baseline quadratic objective. These same values were retained across both case studies to emphasize transferability rather than case-specific retuning.

3.2.2. Replay Buffer with Recency-Adjusted Mixed Sampling. In the online phase, the closed-loop data distribution is time-varying due to set point changes, unmeasured disturbances, and slow plant drift. In addition, the RL agent does not observe the true plant state. It observes estimated states from an identified linear model and observer, which can become less representative as operating conditions change. As a result, older transitions can remain in the replay buffer even though they reflect different regimes or outdated policies. This motivates a replay strategy that supports fast adaptation while preserving broad coverage across set points and operational conditions.

3.2.2.1. Stored Transition and Metadata. Each online interaction appends the tuple

$$(s_t, u_t, r_t, s_{t+1})$$

with $s_t = [\tilde{x}_t^T y_{\text{SP},t}^T u_{t-1}^T]^T$. The action u_t is the applied control move projected onto $\mathcal{U}$, so the critic trains on executed actions. Each stored index i also tracks its insertion step (birth time) b_i , initialized as

$$b_i \leftarrow t$$

3.2.2.2. Minibatch Composition. Given a batch size B , we form a mixed minibatch with

$$B_{\text{per}} = \varphi_{\text{per}} B, \quad B_{\text{rec}} = \varphi_{\text{rec}} B, \\ B_{\text{uni}} = (1 - \varphi_{\text{per}} - \varphi_{\text{rec}}) B$$

where $\phi_{\text{per}}, \phi_{\text{rec}} \in [0, 1]$ are fixed fractions. The final minibatch is $\mathcal{B} = \mathcal{B}_{\text{per}} \cup \mathcal{B}_{\text{rec}} \cup \mathcal{B}_{\text{uni}}$.

3.2.2.3. Prioritized Experience Replay (PER) Component. Each stored index $i \in \{1, \dots, N\}$ carries a nonnegative priority p_i . PER samples indices according to

$$P_{\text{per}}(i) = \frac{(p_i + \epsilon)^\alpha}{\sum_{j=1}^N (p_j + \epsilon)^\alpha}, \quad \alpha \in [0, 1], \quad \epsilon > 0 \quad (41)$$

where α controls the strength of prioritization and ϵ prevents zero-probability sampling.⁴⁶ Setting $\alpha = 0$ recovers uniform sampling.

3.2.2.4. Recent-Window Component. Motivated by recency-bias ideas in replay sampling⁴⁷ let $\mathcal{R}_t$ denote the set of the most recent W indices in the buffer, defined by the W largest birth times $\{b_i\}$. We sample the recent subset uniformly from this window

$$P_{\text{rec}}(i) = \begin{cases} \frac{1}{|\mathcal{R}_t|}, & i \in \mathcal{R}_t \\ 0, & i \notin \mathcal{R}_t \end{cases} \quad (42)$$

which biases updates toward data collected under the most recent operating conditions.

3.2.2.5. Uniform Remainder. The uniform subset $\mathcal{B}_{\text{uni}}$ is drawn uniformly from the entire buffer, with

$$P_{\text{uni}}(i) = \frac{1}{N}$$

3.2.2.6. Mixture Probability and Importance Weights. The overall sampling probability is the mixture

$$q(i) = \phi_{\text{per}} P_{\text{per}}(i) + \phi_{\text{rec}} P_{\text{rec}}(i) + (1 - \phi_{\text{per}} - \phi_{\text{rec}}) P_{\text{uni}}(i) \quad (43)$$

3.2.2.7. Importance-Sampling Weights. To reduce the bias introduced by PER, we apply importance-sampling (IS) weights to the PER subset only. For $i \in \mathcal{B}_{\text{per}}$, we use

$$w_i = \frac{(NP_{\text{per}}(i))^{-\beta}}{\max_{j \in \mathcal{B}_{\text{per}}} (NP_{\text{per}}(j))^{-\beta}}, \quad \beta \in [\beta_0, 1] \quad (44)$$

and set $w_i = 1$ for $i \in \mathcal{B}_{\text{rec}} \cup \mathcal{B}_{\text{uni}}$. The IS exponent β is annealed linearly from β_0 to 1 over a fixed number of sampling steps, as in.⁴⁶

3.2.2.8. Priority Updates. After each critic update, priorities are updated using the average absolute TD error from both critics

$$p_i \leftarrow \text{clip}(\bar{\delta}_i + \epsilon, p_{\min}, p_{\max}), \quad \bar{\delta}_i = \frac{1}{2}(|t_i - Q_{\phi_1}(s_i, u_i)| + |t_i - Q_{\phi_2}(s_i, u_i)|) \quad (45)$$

where t_i is the TD3 target used in the critic update, and $\text{clip}(\cdot)$ prevents extreme priority skew.

Unless stated otherwise, these hyperparameters were kept fixed across all case studies. The values used in the simulations are summarized in Table 1. The hyperparameters in Table 1 were selected using a practical heuristic procedure rather than formal hyperparameter optimization. The TD3 and replay settings were chosen from commonly used values in the deep RL literature and then adjusted only to ensure stable training in preliminary runs. More specifically, the PER parameters $\alpha, \epsilon, \beta_0$,

β_{end} , and N_β were initialized from standard PER practice and then refined only to avoid overly sharp prioritization or unstable critic updates. The recent-window size W was chosen to cover multiple recent set point segments so that the recency subset reflects the current operating regime without collapsing to only the most recent few samples. The mixture fractions $\phi_{\text{per}} = 0.5$ and $\phi_{\text{rec}} = 0.2$ were selected so that PER and recency each have a meaningful role, while the remaining 30% of the minibatch is still drawn uniformly to preserve broader coverage across set point changes and operating conditions. The replay capacity N was chosen as a medium-sized buffer, that is, a fraction of the total online trajectory length, so that it retains data from multiple operating conditions while limiting excessive staleness. After this initial selection, these settings were kept fixed across the case studies to evaluate the proposed framework under a common configuration rather than through case-specific retuning. Similarly, the actor and critic networks were chosen to be relatively large because the offline pretraining data set contains approximately 3×10^6 transitions, so the main concern in pretraining was sufficient approximation capacity over a broad operating range rather than aggressive model-size reduction.

Remark 3 In process control, uniform replay can slow adaptation because many samples reflect older operating regimes or outdated policies. A recency-only window can adapt quickly, but it can also reduce coverage and accelerate forgetting, especially for rare events such as constraint activation or large set point moves. Prioritized experience replay (PER) can also be misleading in this setting. When the agent state is estimated from an identified linear model and the plant drifts or faces unseen disturbances, older transitions may exhibit large TD errors because they are no longer representative of the current regime. PER may then oversample these stale transitions and bias critic updates toward outdated targets. The PER–Recent mixture balances these effects by combining (i) PER to focus updates on high-error transitions, (ii) a recent window to track the current regime, and (iii) uniform sampling to preserve long-range coverage.

Remark 4 Under plant drift, very large buffers can mix data from multiple regimes and increase the spread of TD errors. This can destabilize critic updates when PER becomes too sharp. In our implementation, clipping priorities and keeping a nontrivial recent and uniform fraction reduces this failure mode while retaining diversity across operating conditions.

Remark 5 In RL, buffer size refers to the replay capacity N , that is, the number of stored transitions.⁴⁸ A buffer is very large when N is comparable to the full training history, so the oldest samples may come from operating regimes and policies that are far from the current closed-loop behavior. A buffer is medium-sized when N spans only a fraction of the training history but still covers multiple set point changes and operational changes. This choice limits staleness and reduces policy and regime mismatch in the replayed data, which is important in time-varying operating conditions. Large replay buffers have been reported to degrade learning in such settings.^{47–49}

Remark 6 Reward shaping and replay design target different failure modes. The shaped reward primarily reduces steady-state offset, whereas the PER–Recent replay primarily improves adaptation under time-varying operating conditions by emphasizing recent data while retaining broader coverage. Combining both mechanisms yields the most consistent behavior across scenarios, so they are presented together as an online fine-tuning framework.

Algorithm 2 summarizes the online fine-tuning loop, which augments the pretrained TD3 agent with offset-aware reward shaping and PER–Recent mixed-replay.

Algorithm 2: Online fine-tuning with reward shaping and PER–Recent mixed-replay

Input: Pretrained actor μ_θ , critics Q_{ϕ_1}, Q_{ϕ_2} , target networks, replay buffer $\mathcal{D}$,

reward shaping parameters, TD3 parameters, replay fractions $(\varphi_{\text{per}}, \varphi_{\text{rec}})$,

recent window size W , PER parameters (α, ε) , IS schedule $(\beta_0, \beta_{\text{end}}, N_\beta)$.

```

1 for time step  $t = 0, 1, \dots$  do
2   Form  $s_t = [\tilde{x}_t^\top \ y_{SP,t}^\top \ u_{t-1}^\top]^\top$ 
3   Compute  $u_t = \Pi_{\mathcal{U}}(\mu_\theta(s_t))$  and apply it to the plant
4   Observe  $s_{t+1}$  and compute  $r_t$  using the shaped reward (Eq. (39))
5   Store  $(s_t, u_t, r_t, s_{t+1})$  in  $\mathcal{D}$ , set the birth time  $b_i \leftarrow t$ , and initialize  $p_i \leftarrow p_{\text{max}}$ 
6   Sample  $\mathcal{B}_{\text{per}}$  from  $P_{\text{per}}(i)$  (Eq. (41))
7   Sample  $\mathcal{B}_{\text{rec}}$  uniformly from the recent window  $\mathcal{R}_t$  (Eq. (42))
8   Sample  $\mathcal{B}_{\text{uni}}$  uniformly from the full buffer
9   Form  $\mathcal{B} = \mathcal{B}_{\text{per}} \cup \mathcal{B}_{\text{rec}} \cup \mathcal{B}_{\text{uni}}$ 
10  Compute IS weights  $w_i$  for  $i \in \mathcal{B}_{\text{per}}$  (Eq. (44)); set  $w_i = 1$  otherwise
11  Update critics using weighted TD errors (use  $w_i$  as per-sample multipliers)
12  Update priorities for sampled indices using Eq. (45)
13  if update delay condition is met (every  $d$  steps) then
14    Update actor using the TD3 policy gradient loss
15    Soft-update target networks
16  end
17 end

```

Remark 7 The proposed workflow assumes only the linear model that is already available in typical plant controller designs, that is, an identified state-space (or step-response) model obtained from routine step tests, together with input bounds and the normal set point operating range. No additional first-principles modeling, high-fidelity simulator, or robust NMPC design is introduced. Offline pretraining data are generated using the same offset-free linear MPC formulation built on this identified linear model, and the TD3 policy is initialized from those MPC trajectories. As a result, the method targets settings where only the standard MPC identification model is available, unlike approaches that rely on a high-fidelity simulator or a carefully designed stabilizing backup controller for online supervision.

3.2.3. Episode Definition and Evaluation Protocol. All closed-loop simulations are performed in continuous operation. In keeping with practicality, the plant is not “reset” when the set point changes. For reporting purposes only, we define an episode as one full cycle of a repeating two-segment set point schedule. In episode $k = 1, 2, \dots$, let $\tau = 1, \dots, L$ denote the within-episode time index, where $L = L_a + L_b$. Segment A holds the constant set point $\mathbf{y}_{\text{sp}}^{(a)}$ for L_a steps, and segment B holds the constant set point $\mathbf{y}_{\text{sp}}^{(b)}$ for L_b steps. The set point schedule within each episode is

$$\mathbf{y}_{\text{sp}}(k, \tau) = \begin{cases} \mathbf{y}_{\text{sp}}^{(a)}, & \tau = 1, \dots, L_a, \\ \mathbf{y}_{\text{sp}}^{(b)}, & \tau = L_a + 1, \dots, L. \end{cases} \quad (46)$$

This schedule repeats from episode to episode, while the plant and observer states carry over.

We report the episode-average reward as

$$\bar{r}_k = \frac{1}{L} \sum_{\tau=1}^L r(k, \tau) \quad (47)$$

where $r(k, \tau)$ is the per-step shaped reward used during online learning. Only the counters used to compute $\bar{r}_k$ are reset at episode boundaries. The plant and observer states are not reset.

All controller comparisons use the same set point schedule in (46), the same disturbance profiles, the same sampling time, and the same actuator bounds. Reported results are averaged over multiple runs with different random seeds. During online operation, the agent continues updating its networks across episodes using the accumulated replay buffer. The episode boundary is used only for reporting and does not terminate learning.

3.2.3.1. Evaluation Episode. After the final online-training episode, performance is assessed in evaluation mode. In this mode, network updates are disabled and the policy is executed without exploration noise. Actions are generated using the target actor network rather than the online actor. The target actor is a smoothed version of the online actor because it is updated through a soft-update rule (Polyak averaging), which reduces rapid parameter changes and typically yields more stable actions in closed loop. As a result, the episode-average reward in evaluation can be higher than the rewards observed during online training, since it reflects noise-free execution with a more averaged policy. In reward plots, this can appear as a spike in the last episode and should be interpreted as the evaluation result rather than a sudden improvement in learning.

3.2.4. Online Warm-Start Strategy. Direct online training on a large-scale process can be unsafe if the policy is updated too early or if exploration dominates before the value estimates become reliable. To reduce this risk, we use a staged warm-start schedule during online deployment. The key idea is to start from an MPC-like policy (from the imitation phase), collect closed-loop data under safe actions, and delay policy updates until the critic is better calibrated on on-policy data. In the case studies below, this staged warm-start procedure is used as a common deployment safeguard for the MPC-pretrained TD3 controllers considered in this work. It should therefore be interpreted as part of the online deployment strategy, rather than as a separate controller variant intended for an independent performance comparison.

Phase 0: safe rollout without learning (buffer warm-up). For the first N_{warm} episodes, the controller uses the pretrained actor without any policy learning. Actions are always projected onto the input bounds $\mathcal{U}$, and the replay buffer is populated with transitions from closed-loop operation. This stage provides two benefits. First, it keeps the early behavior close to the OF-MPC policy learned during pretraining. Second, it prevents unstable updates when the buffer is still small and unrepresentative.

Phase 1: critic-only adaptation (actor frozen). After the buffer warm-up, we freeze the actor parameters and update only the critics for N_{crit} episodes using the shaped online reward. During this stage, the critic learns value estimates that are consistent with the current closed-loop trajectories and the online objective. Freezing the actor avoids abrupt policy shifts while the critic is still adjusting.

Phase 2: full online fine-tuning. Once the critic has adapted, we enable standard TD3 updates for both actor and critics. The applied input is always projected onto $\mathcal{U}$. In this final stage, the shaped reward and mixed-replay strategy drive the policy toward improved transient response and reduced steady-state error, while the warm-started initialization helps maintain safe and stable operation.

3.2.4.1. Implementation Note. The warm-start period shown in the closed-loop plots corresponds to Phases 0 and 1, where learning is delayed or restricted to critic updates. This schedule is used to make online deployment safer while still allowing adaptation to plant nonlinearities and disturbances.

The controller comparisons in this work are intended to isolate two questions. First, the comparison between OF-MPC and RL1 evaluates the effect of MPC-based pretraining followed by online fine-tuning using the baseline one-step MPC-style reward. Second, the comparison between RL1 and RL2 evaluates the proposed online modifications, since both agents use the same offline data set, pretrained initialization, TD3 architecture, and staged warm-start schedule, and differ only in

the online reward and replay mechanism. In this study, the warm-start strategy is used as a common safety-oriented deployment procedure rather than as a separate controller variant. A finer ablation between reward shaping and replay would be useful, but is beyond the scope of the present work.

During online training, only small exploration noise was used because the actor was already initialized by OF-MPC imitation and the goal of the online phase was local refinement of an MPC-like policy rather than broad exploratory search. This choice was also consistent with the safety-oriented warm-start strategy used during deployment.

3.3. Illustrative Example: Styrene Polymer Reactor

The styrene polymerization reactor shown in Figure 5.⁵⁰ is used as an illustrative example for the proposed approach. This process is a nonlinear multi-input multioutput (MIMO) system with strong coupling between variables, making it a realistic and challenging scenario to illustrate advanced control methods. The manipulated inputs are the monomer flow rate Q_M and the cooling fluid flow rate Q_c . The controlled outputs are the reactor temperature T and the intrinsic viscosity η . A detailed description of the polymerization reactor is provided in Supporting Information Section S5.

3.3.1. Performance under Nominal Conditions. The linear state-space model described in Supporting Information Section S5 is used to implement the OF-MPC controller described in Section 2.2. The OF-MPC tracking performance is included in Figures S6 and S7 as a reference baseline. The controller settings and the online reward hyperparameters used in this case study are summarized in Table 2. The shaped-reward hyperparameters were selected heuristically based on the structure of eqs 27–39. The transition-width parameter τ_{frac} was chosen so that the in-band gate changes smoothly across the band boundary rather than switching too sharply at $z_i = 1$. The linear shaping weights were kept modest and symmetric, with $\gamma_{\text{out}} = \gamma_{\text{in}}$, so that the additional shaping remains visible both inside and outside the band without dominating the baseline quadratic penalty. The in-band bonus parameters β and k_b were then chosen to strengthen the learning signal for removing small residual errors while keeping the bonus smooth near the band edge. In particular, these values were selected to avoid an overly sharp increase in reward inside the band, which could otherwise encourage oscillatory behavior or distort the baseline MPC-style objective. The same values were retained in both case studies to emphasize transferability of the proposed online modification rather than case-specific retuning. A systematic sensitivity analysis of these hyperparameters was not pursued in this study and remains a useful direction for future work.

The OF-MPC structure, weights, and observer gains were kept fixed across all scenarios to reflect typical industrial practice, where frequent retuning is avoided. Only the online learning components (policy updates driven by the shaped reward and replay) adapted during training. All code, configuration files, and postprocessing scripts for the polymer reactor case study are available in the project repository on GitHub. The online computational cost of the polymer benchmark was modest on the hardware used in this study. A complete online run of 200 episodes required approximately 20 min of wall-clock time, including ODE integration, environment stepping, data collection, replay insertion, and TD3 updates. Since the control action is applied every 30 min in process time, the computation required per control move is negligible relative to the sampling interval. In all reported polymer reactor simulations, only

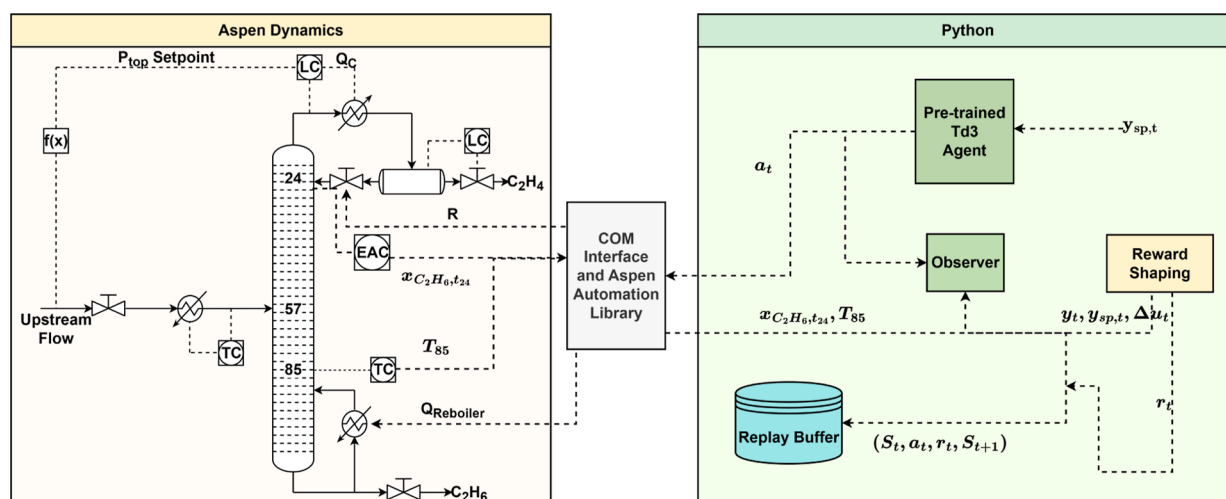


Figure 15. Closed-loop implementation of the Aspen Dynamics C_2 splitter and the TD3 controller. Aspen Dynamics advances the rigorous column model and communicates measurements and input moves with Python through a COM-based automation interface. Python updates the augmented state estimator (offset-free structure), forms the RL state, computes the control action using the TD3 policy, and stores transitions in the replay buffer for online fine-tuning.

actuator bounds were enforced. At each time step, the applied input was projected onto the admissible set $\mathcal{U}$, and no actuator bound violations were observed. All polymer RL experiments used a discount factor of $\gamma = 0.99$. The pretrained RL agents are first evaluated on the reactor under nominal conditions. RL_1 uses the one step MPC objective function as the online reward, while RL_2 uses the proposed modifications (offset-aware reward shaping and mixed-replay).

The controller comparisons in this work are intended to isolate two questions. First, the comparison between OF-MPC and RL_1 evaluates the effect of MPC-based pretraining followed by online fine-tuning using the baseline one-step MPC-style reward. Second, the comparison between RL_1 and RL_2 evaluates the proposed online modifications, since both agents use the same offline data set, pretrained initialization, TD3 architecture, and staged warm-start procedure, and differ only in the online reward and replay mechanism. A finer ablation separating the individual effects of reward shaping and mixed replay would be useful, but is beyond the scope of the present work.

Figure 6 shows the results for RL_1 . During online training, the episode-average reward (averaged over five runs) improves after the warm-start period and remains above the OF-MPC baseline. In the full-horizon closed-loop trajectory, RL_1 maintains stable operation under repeated set point changes. In the evaluation episode without exploration noise, RL_1 reaches the set points faster than OF-MPC. However, a small steady-state offset remains, which is quantified in Table 3.

Figure 7 shows the performance of RL_2 . Compared to RL_1 , RL_2 maintains the improved transient response while further reducing the steady-state error, as summarized in Table 3. The episode-average reward (averaged over five runs) remains above the OF-MPC baseline. In the evaluation episode without exploration noise, RL_2 tracks both set points with fast settling and smaller steady-state error than RL_1 .

Figure 8 compares RL_1 , RL_2 , and OF-MPC in the nominal evaluation episode with exploration noise disabled. Both RL policies reach the new set points faster than OF-MPC. Compared to RL_1 , RL_2 preserves the fast transient response and settles closer to the set points in both outputs, which leads to

the smallest steady-state error. This improvement is consistent across runs, as also reflected by the steady-state error statistics in Table 3.

3.3.2. Performance under Operational Changes. To evaluate robustness to disturbances, slowly varying operating changes are introduced, as shown in Figure 9. The heat-transfer coefficient factor hA is decreased by 10% until the midpoint of the run to represent fouling, and then held constant. In addition, the initiator feed rate Q_i is decreased linearly by 5% over time, while the solvent feed rate Q_s is increased linearly by 5%. These changes are not included in the model used for pretraining and therefore represent unseen plant drift.

The OF-MPC baseline is evaluated under the same operating changes. Its long-run response and last-episode performance are provided in Figure S7.

Figure 10 shows the results for RL_1 under operational changes (mean of five independent runs). The episode-average reward improves after warm-start and remains above the OF-MPC baseline. In the evaluation episode with exploration noise disabled, RL_1 still reaches the new set points faster than OF-MPC, which reflects the benefit of MPC-based pretraining. However, a small steady-state offset persists, as summarized in Table 3.

Figure 11 reports the corresponding results for RL_2 under operational changes (mean of five independent runs). RL_2 maintains the fast transient response observed in the nominal case while compensating steady-state error more effectively than RL_1 . The improvement is consistent across runs, and the reward remains reliably above the OF-MPC baseline after warm-start period. The steady-state errors are summarized in Table 3.

Figure 12 compares all three controllers in the last evaluation episode with exploration noise disabled. Both RL agents settle faster than OF-MPC, while RL_2 achieves the smallest steady-state error and shows the most consistent behavior.

Overall, the results show that MPC-based pretraining improves transient response relative to OF-MPC in both nominal conditions and under operational changes. Table 3 shows that RL_2 reduces steady-state error substantially more than RL_1 in both outputs and both set point changes, which supports the role of the offset-aware reward shaping and mixed-

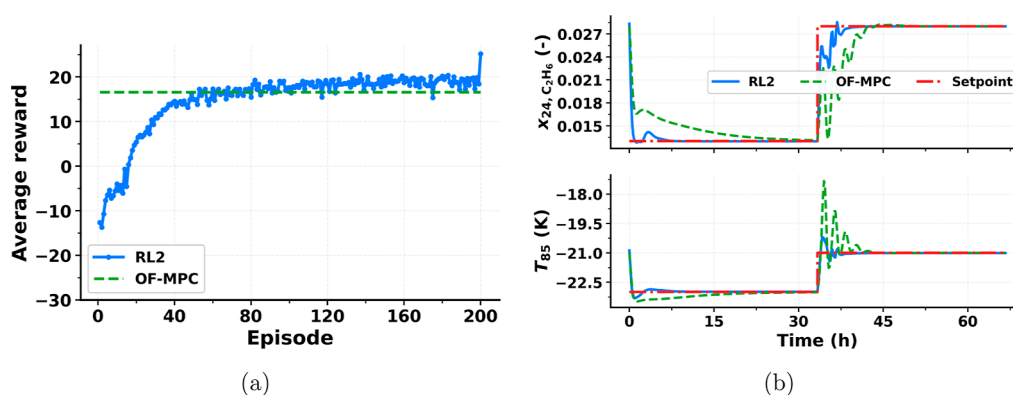


Figure 16. C_2 splitter under nominal conditions. (a) Episode-average reward during online training. The solid line with markers shows the RL episode-average reward, and the dashed horizontal line indicates the OF-MPC baseline. (b) Closed-loop tracking of x_{24, C_2H_6} and T_{85} in the last evaluation episode with exploration noise disabled. RL settles faster than OF-MPC, while the stepwise dash-dot line denotes the set point sequence.

replay strategy introduced in Section 3. In addition, Figure 13 shows the distribution of the absolute scaled steady-state error across runs. Across both scenarios, RL_2 has lower medians and tighter spreads than RL_1 , which indicates more reliable and repeatable steady-state tracking. To further compare the final tracking behavior of RL_2 across scenarios, Figure 14 directly compares the scaled steady-state absolute error distributions under nominal and operational-change conditions. The results show that the final $|e|$ values remain low in both cases and are comparable in magnitude. The operational-change case does not show a systematic improvement over the nominal case. Instead, some output and set point combinations are similar or slightly smaller, while others show a larger spread under operational changes.

4. RESULTS FOR THE INDUSTRIAL-SCALE ETHYLENE SPLITTER

Section 2 introduced the industrial-scale ethylene splitter (C_2 splitter) used in this work (Figure 1). Here, we evaluate the

Table 4. Parameter Settings for OF-MPC and the Online Shaped Reward (C_2 Splitter Case Study)

OF-MPC		modified reward function	
parameter	value	parameter	value
Q (diag)	(1, 1)	τ_{frac}	0.7
R (diag)	(1, 1)	$\gamma_{\text{out}} = \gamma_{\text{in}}$	0.5
prediction horizon N_p	9	β	7
control horizon N_c	3	k_b	12

proposed RL control framework on the rigorous Aspen Dynamics model. The control objective is the set point tracking of (i) the ethane impurity mole fraction on tray 24, x_{24, C_2H_6} , and (ii) the tray 85 temperature, T_{85} , using two manipulated variables (reflux flow and reboiler duty). The baseline controller is the linear OF-MPC used throughout this study. The RL policy is initialized by MPC-imitation pretraining and then fine-tuned online using the proposed shaped reward and mixed-replay sampling strategy.

Figure 15 summarizes the closed-loop implementation. Aspen Dynamics advances the high-fidelity column model and returns measured outputs to Python through a COM-based automation interface. In Python, state estimation is performed using the augmented linear model, the RL state is formed, the TD3 policy

is evaluated, and the computed input moves are sent back to Aspen Dynamics at each sampling instant. The resulting transitions are stored in the replay buffer for online fine-tuning using the proposed sampling design.

4.1. Nominal Set Point Tracking

Figure 16 compares the episode-average reward during online training and the closed-loop outputs in the last evaluation episode. The set point profile includes an initial hold followed by a simultaneous step change in both outputs. A standalone OF-MPC response for this nominal scenario is provided in Figure S2. The controller settings and the online training hyperparameters used in this case study are summarized in Table 4. The OF-MPC structure, weights, and observer gains were kept fixed across all scenarios to reflect typical industrial practice, where frequent retuning is avoided. Only the online learning components, namely the policy updates driven by the shaped reward and replay mechanism, adapted during training. All code, Aspen Dynamics interface scripts, and analysis notebooks for the C_2 splitter case study are available in the project repository on GitHub. In addition, the online computation remained well below the available control interval. For the C_2 splitter case in Aspen Dynamics, each control step required approximately 1 s on average, including COM communication, plant advancement, data collection, policy evaluation, replay-buffer operations, and gradient updates. Since the control action is updated every 10 min in the process, the online wall-clock cost is significantly smaller than the available control window. The reported offline pretraining time of about 12 h is a one-time offline cost and should therefore be considered separately from online real-time feasibility. In all reported C_2 splitter simulations in Aspen Dynamics, only actuator bounds were enforced. At each time step, the applied input was projected onto the admissible set $\mathcal{U}$, and no actuator bound violations were observed. All C_2 splitter RL experiments used a discount factor of $\gamma = 0.99$. In this nominal case, the RL policy reaches the initial set points faster than OF-MPC, most clearly in x_{24, C_2H_6} . After the step change, RL settles faster in both outputs, whereas OF-MPC exhibits a longer settling phase, again most noticeably in the composition. Overall, the learned policy improves transient performance on the rigorous Aspen Dynamics model while maintaining stable set point tracking. The reward increases rapidly in early episodes and then gradually refines, remaining above the OF-MPC baseline in late training.

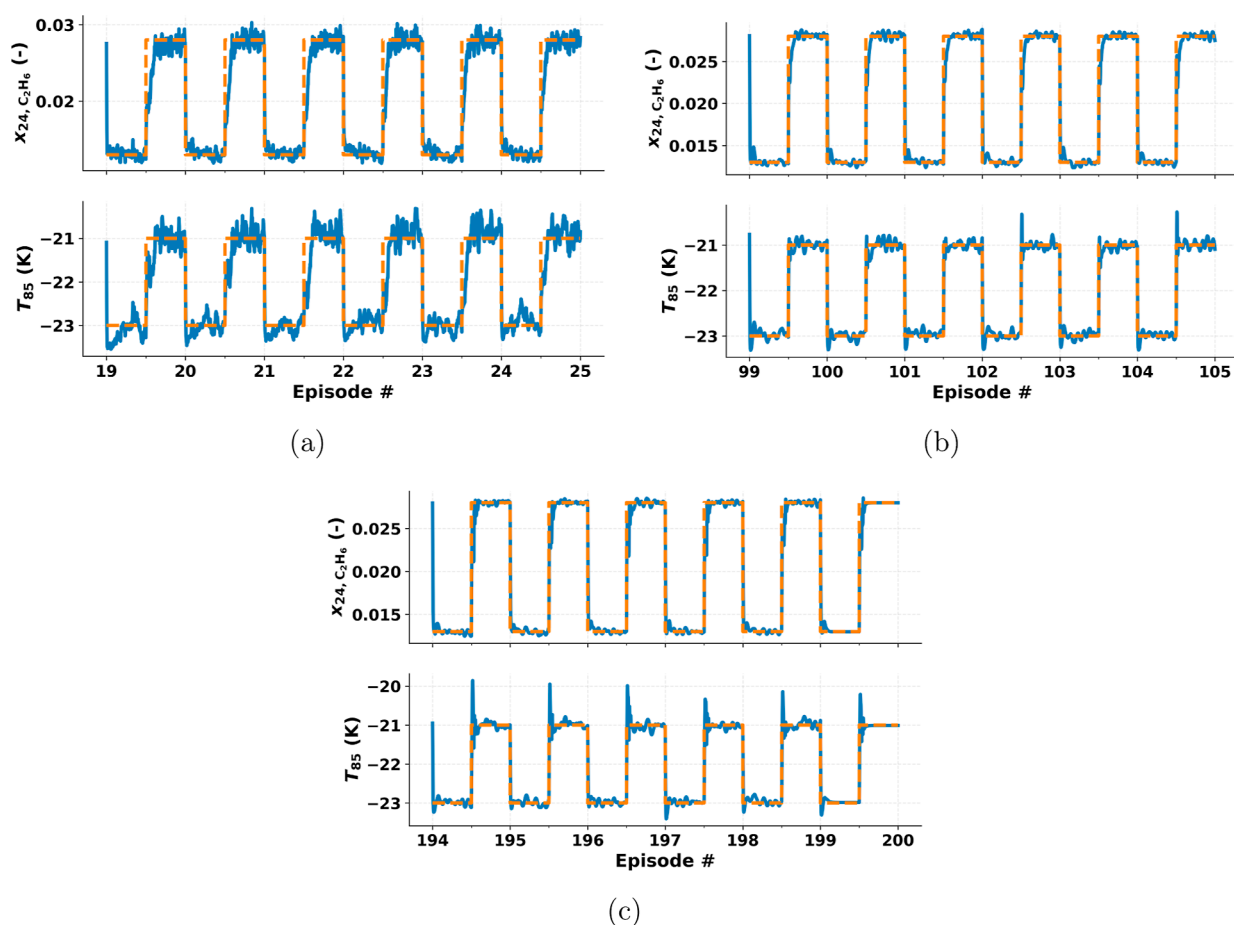


Figure 17. RL closed-loop tracking shown over three training windows. Solid lines show the RL outputs x_{24, C_2H_6} and T_{85} , and dashed lines show the set points. (a) Early training window (episodes 20–25), showing stable and reasonable tracking even when the episode-average reward is still lower due to exploration and transient penalties. (b) Mid training window (episodes 100–105), where tracking becomes faster and more consistent across set points. (c) Late training window (episodes 195–200), including the final evaluation episode, which yields the smoothest trajectories and the best tracking among the shown windows.

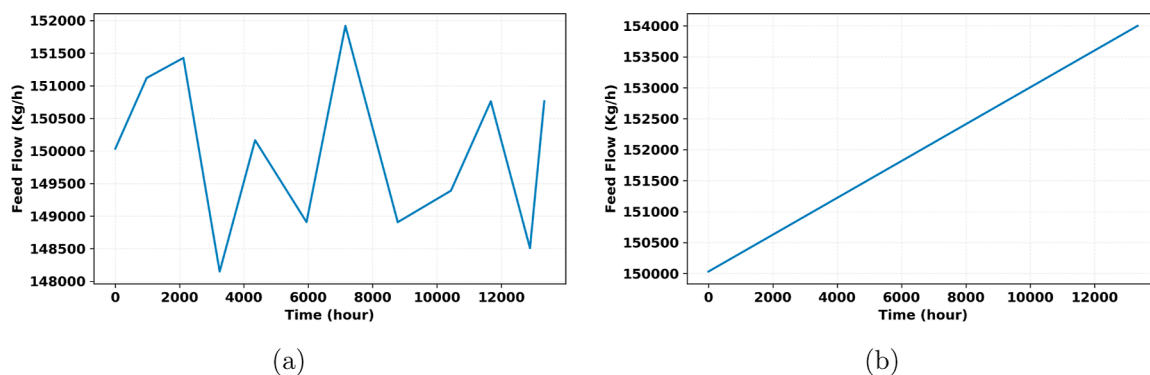


Figure 18. C_2 splitter operation change profiles used for evaluation. (a) Scenario 1: fluctuations in the feed flow rate. (b) Scenario 2: gradual ramp in the feed flow rate.

Figure 17 illustrates how the closed-loop performance evolves over training. In the early window (episodes 20–25), tracking is already stable and follows the set point changes with reasonable settling, even though the episode-average reward is still relatively low due to ongoing exploration. By the mid training window (episodes 100–105), the response becomes faster and more consistent across set point changes, with smaller explorations around each set point. In the late window (episodes 195–200), tracking remains tight over set point changes and the residual

offset is further reduced. The final episode is executed in evaluation mode, where learning updates and exploration noise are disabled and the target actor is used, and it yields the smoothest trajectories and the best tracking among the shown windows.

4.2. Performance under Plant Operation Changes

In industrial operation, set point changes are not the only source of disturbances. Variations in upstream units, production

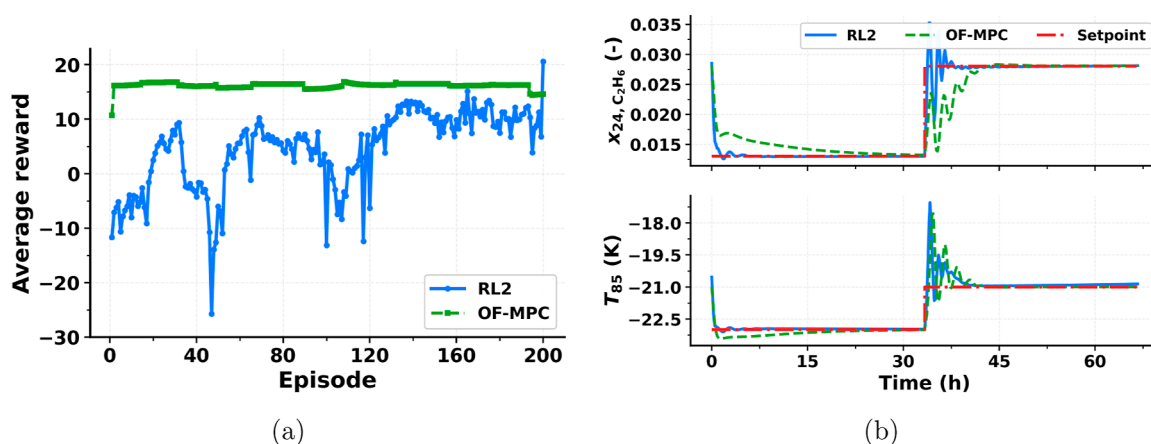


Figure 19. C_2 splitter under operation changes with throughput fluctuations. (a) Episode-average reward during online training. The solid line shows the RL episode-average reward, and the dashed line indicates the OF-MPC baseline. (b) Closed-loop tracking of x_{24, C_2H_6} and T_{85} in the last evaluation episode. RL settles faster than OF-MPC, while the stepwise dash-dot line denotes the set point sequence.

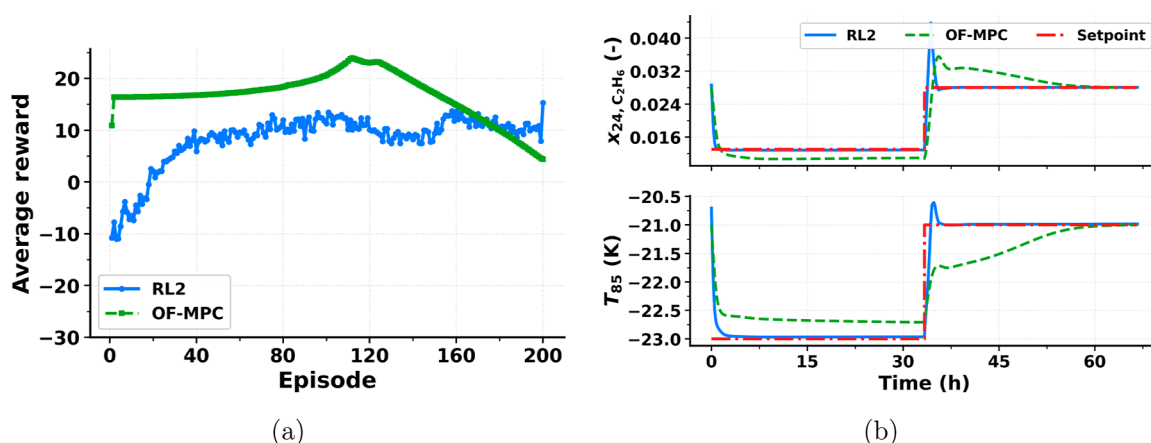


Figure 20. C_2 splitter under operation changes. (a) Episode-average reward during online training. The solid line shows the RL episode-average reward, and the dashed line indicates the OF-MPC baseline. (b) Closed-loop tracking of x_{24, C_2H_6} and T_{85} in the last evaluation episode. RL settles faster than OF-MPC, while the stepwise dash-dot line denotes the set point sequence.

targets, and routine operating adjustments introduce both slow drift and discrete shifts in plant conditions. To evaluate robustness in a more realistic setting, we impose time-varying operation changes through the feed flow rate profiles in Figure 18. Two scenarios are considered: (i) fluctuation changes with multiple operating shifts and (ii) a gradual ramp over the full horizon.

4.2.1. Operation Change Scenarios. Figure 18 summarizes the two feed flow profiles used for evaluation. Scenario 1 applies fluctuations, where the feed flow rate is adjusted multiple times to represent fluctuations in upstream behavior. Scenario 2 applies a gradual ramp in the feed flow rate to represent slow drift in operating targets or upstream availability. In both cases, the controller tracks the same output set points, which enables a direct comparison between the RL policy and OF-MPC under changing plant conditions.

4.2.2. Fluctuation Scenario. Figure 19 shows closed-loop performance when the feed flow rate follows a ramp profile with fluctuations. Relative to OF-MPC, the RL policy settles faster after the set point change and maintains improved tracking under the varying conditions. In the last evaluation episode shown in Figure 19b, the RL policy achieves higher performance than the OF-MPC baseline, indicating that, without exploration noise, the learned policy can drive the plant toward the desired

outputs more effectively than OF-MPC. During online fine-tuning, the episode-average reward remains close to the OF-MPC reference (Figure 19a), suggesting that performance is not jeopardized while the agent adapts to time-varying operating conditions. Once exploration is turned off and target policy is executed deterministically, the RL controller outperformed OF-MPC in this scenario. The reward curve is also more variable than in the nominal case, which reflects the increased difficulty of this scenario. A standalone OF-MPC baseline for this scenario is provided in Figure S3.

4.2.3. Ramp Operation Change. Figure 20 shows closed-loop performance under a gradual ramp in feed flow rate. As the operating point drifts, the OF-MPC baseline tracking performance degrades because the fixed linear prediction model becomes less representative of the changing plant conditions. In contrast, the RL policy maintains tighter regulation through the drift and reaches steady-state conditions significantly faster. This scenario is intentionally selected to be challenging for OF-MPC within the evaluation horizon: over 200 samples, OF-MPC may require substantially more time to converge, while the RL policy achieves near steady-state tracking within the same horizon.

The reward curve indicates that RL performance improves over training and remains close to the OF-MPC reference in late

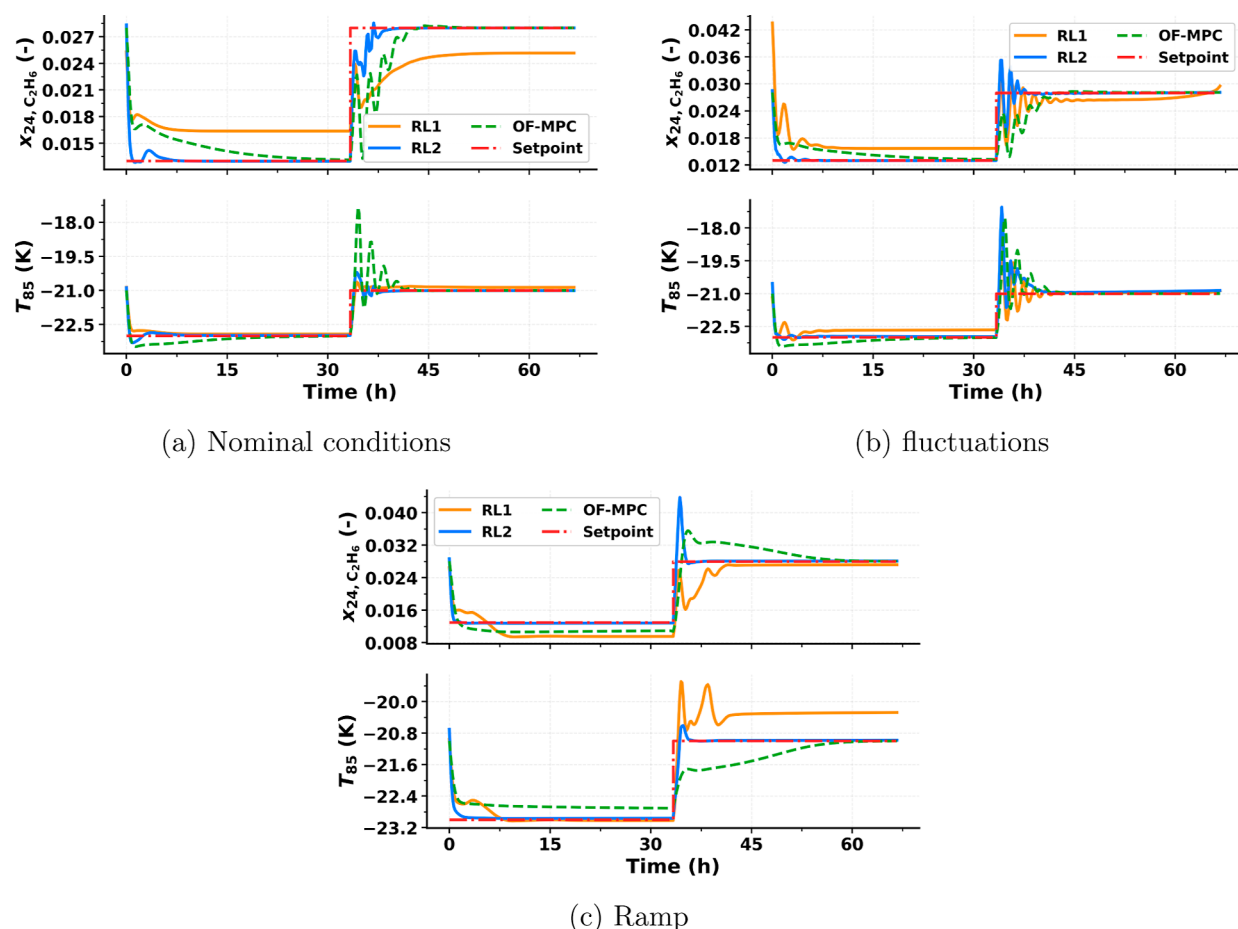


Figure 21. C_2 splitter output comparison in the last evaluation episode. (a) Nominal conditions, (b) operation changes with throughput fluctuations, and (c) Ramp operation change. Across all scenarios, RL_2 tracks the set points with faster settling than OF-MPC and smaller steady-state error than RL_1 . The stepwise dash-dot line denotes the set point sequence for x_{24,C_2H_6} and T_{85} .

Table 5. Steady-State Absolute Tracking Error at the End of the Evaluation Episode^a

	nominal		ramp with fluctuations		ramp	
	SP1	SP2	SP1	SP2	SP1	SP2
x_{24,C_2H_6}						
RL_1	0.00336	0.00285	0.00267	0.00060	0.00348	0.00083
RL_2	0.00003	0.00001	0.00002	0.00006	0.00018	0.00007
reduction (%)	99.16	99.63	99.16	89.69	94.89	91.05
T_{85} [K]						
RL_1	0.07441	0.13431	0.33396	0.12027	0.03546	0.71852
RL_2	0.01391	0.01222	0.02802	0.04705	0.02278	0.01604
reduction (%)	81.31	90.90	91.61	60.87	35.78	97.77

^aReduction is computed as $100\left(1 - \frac{RL_2}{RL_1}\right)$.

episodes, suggesting that online fine-tuning does not jeopardize closed-loop behavior under operating drift. After sufficient training, especially toward the end, the RL episode-average reward remains consistently above the OF-MPC baseline, indicating improved closed-loop performance relative to OF-MPC. A standalone OF-MPC baseline for this scenario is provided in Figure S4.

4.3. Comparison across Operating Scenarios

This subsection compares the closed-loop output trajectories of RL_1 , RL_2 , and OF-MPC across the nominal, ramp with fluctuations, and gradual ramp scenarios. The purpose is to isolate the effect of the proposed modifications in RL_2 relative to

the MPC-imitation baseline RL_1 and to assess whether the learned policies maintain stable tracking on the rigorous Aspen Dynamics model under different operating conditions. Figure 21 summarizes the last evaluation episode for each scenario, and Table 5 reports the corresponding steady-state absolute tracking errors.

In the nominal case (Figure 21a), both RL policies settle faster than OF-MPC. Compared to RL_1 , RL_2 settles closer to the set points for both x_{24,C_2H_6} and T_{85} , which indicates improved steady-state accuracy. Under the ramp with fluctuations scenario (Figure 21b), both RL policies maintain stable set point tracking and settle faster than OF-MPC. In addition, RL_2 achieves

smaller steady-state error than RL_1 , consistent with Table 5. Under the gradual ramp scenario (Figure 21c), RL_2 again settles faster than OF-MPC and improves steady-state accuracy relative to RL_1 , particularly for T_{85} after the set point change.

Table 5 confirms that the proposed modifications in RL_2 improve steady-state offset elimination across all operating scenarios. The largest gains are for composition tracking, where RL_2 reduces the steady-state error by roughly 90–99% relative to RL_1 across scenarios. Temperature tracking also improves substantially, including under the gradual ramp scenario where RL_1 exhibits a much larger residual error after the set point change.

In summary, the proposed framework provides a practical path to deploy MPC-pretrained RL in process control while improving near offset-free performance during online training. Across both the styrene reactor example and the Aspen Dynamics C_2 splitter, RL_2 consistently improves steady-state tracking relative to RL_1 while preserving the faster settling observed relative to OF-MPC. The improvements persist under disturbances and operating changes, indicating that the combination of offset-aware reward shaping and mixed-replay sampling is effective in time-varying operating conditions where MPC imitation alone may be insufficient.

5. CONCLUSION

This work proposes a practical RL control framework for set point tracking in chemical processes, with the goal of improving performance over a well-designed offset-free linear MPC while maintaining safe and implementable online learning. Building on MPC-imitation pretraining of a TD3 agent, we show that using a one-step quadratic MPC-style reward alone can be insufficient to achieve near offset-free performance in large-scale and time-varying operating conditions. To address this limitation, we propose two online modifications: an offset-aware reward shaping term that increases the learning signal near the set point, and a mixed-replay sampling strategy that combines prioritized replay, a recency component, and uniform sampling to balance adaptation and coverage. The proposed framework is demonstrated on two case studies. For the styrene polymerization reactor, the modified policy RL_2 preserves the transient benefits of RL and reduces steady-state tracking error relative to the MPC-imitation baseline RL_1 , under both nominal conditions and slowly varying disturbances. For the industrial-scale ethylene splitter simulated in Aspen Dynamics and interfaced with Python through a COM-based automation workflow, RL_2 consistently improves steady-state accuracy over RL_1 across nominal operation, fluctuations, and gradual ramps, while maintaining faster settling than the OF-MPC baseline in the evaluated scenarios. These results support the main claim that explicit shaping toward small steady-state corrections, together with replay sampling that emphasizes relevant recent experience without discarding broader coverage, improves online adaptation of MPC-pretrained RL for large-scale process control.

■ ASSOCIATED CONTENT

SI Supporting Information

The Supporting Information is available free of charge at <https://pubs.acs.org/doi/10.1021/acs.iecr.6c00277>.

Additional details on the C_2 splitter model and identification, standalone OF-MPC baseline results for the C_2 splitter under nominal and operating-change

scenarios, the styrene polymerization reactor model and identification, and standalone OF-MPC baseline results for the polymerization reactor under nominal and operation-change scenarios (PDF)

■ AUTHOR INFORMATION

Corresponding Author

Prashant Mhaskar – Department of Chemical Engineering, McMaster University, Hamilton, Ontario L8S 4L7, Canada; orcid.org/0000-0001-5866-211X; Email: mhaskar@mcmaster.ca

Authors

Amir Hossein Hamed – Department of Chemical Engineering, McMaster University, Hamilton, Ontario L8S 4L7, Canada
Hesam Hassanpour – Department of Chemical Engineering, McMaster University, Hamilton, Ontario L8S 4L7, Canada
Ankur Kumar – Smart Operations Technology, Linde plc., Tonawanda, New York 14150, United States
Atharva Vijay Suryavanshi – Linde plc., Tonawanda, New York 14150, United States

Complete contact information is available at: <https://pubs.acs.org/10.1021/acs.iecr.6c00277>

Notes

The authors declare no competing financial interest.

■ ACKNOWLEDGMENTS

Financial support from the Natural Sciences and Engineering Research Council of Canada (NSERC) and the McMaster Advanced Control Consortium (MACC) is gratefully acknowledged.

■ REFERENCES

- (1) Bequette, B. W. *Process Control: Modeling, Design, and Simulation*; Pearson Education, 2023.
- (2) Wahid, A.; Prasetyo, A. A Comparative study between MPC and PI controller to control vacuum distillation unit for producing LVGO, MVGO, and HVGO. *IOP Conf. Ser.: Mater. Sci. Eng.* **2018**, 334, 012020.
- (3) Lee, J. H. Model predictive control: Review of the three decades of development. *Int. J. Control Autom. Syst.* **2011**, 9, 415–424.
- (4) Santos, L. O.; Afonso, P. A.; Castro, J. A.; Oliveira, N. M.; Biegler, L. T. On-line implementation of nonlinear MPC: an experimental case study. *Control Eng. Pract.* **2001**, 9, 847–857.
- (5) Chen, Z.; Henson, M. A.; Belanger, P.; Megan, L. Nonlinear model predictive control of high purity distillation columns for cryogenic air separation. *IEEE Trans. Control Syst. Technol.* **2010**, 18, 811–821.
- (6) Chen, X.; Heidarnejad, M.; Liu, J.; Christofides, P. D. Distributed economic MPC: Application to a nonlinear chemical process network. *J. Process Control* **2012**, 22, 689–699.
- (7) Hassanpour, H.; Corbett, B.; Mhaskar, P. Integrating dynamic neural network models with principal component analysis for adaptive model predictive control. *Chem. Eng. Res. Des.* **2020**, 161, 26–37.
- (8) Jalanko, M.; Sanchez, Y.; Mhaskar, P.; Mahalec, V. Flooding and offset-free nonlinear model predictive control of a high-purity industrial ethylene splitter using a hybrid model. *Comput. Chem. Eng.* **2021**, 155, 107514.
- (9) Hassanpour, H.; Corbett, B.; Mhaskar, P. Artificial neural network-based model predictive control using correlated data. *Ind. Eng. Chem. Res.* **2022**, 61, 3075–3090.
- (10) Luo, J.; Abdullah, F.; Christofides, P. D. Model predictive control of nonlinear processes using neural ordinary differential equation models. *Comput. Chem. Eng.* **2023**, 178, 108367.

- (11) Kheradmandi, M.; Mhaskar, P. Model predictive control with closed-loop re-identification. *Comput. Chem. Eng.* **2018**, *109*, 249–260.
- (12) Hassanpour, H.; Mhaskar, P.; Corbett, B. A practically implementable reinforcement learning control approach by leveraging offset-free model predictive control. *Comput. Chem. Eng.* **2024**, *181*, 108511.
- (13) Daoutidis, P.; Megan, L.; Tang, W. The future of control of process systems. *Comput. Chem. Eng.* **2023**, *178*, 108365.
- (14) Petsagkourakis, P.; Sandoval, I. O.; Bradford, E.; Zhang, D.; del Rio-Chanona, E. A. Reinforcement learning for batch bioprocess optimization. *Comput. Chem. Eng.* **2020**, *133*, 106649.
- (15) Shin, J.; Badgwell, T. A.; Liu, K.-H.; Lee, J. H. Reinforcement learning—overview of recent progress and implications for process control. *Comput. Chem. Eng.* **2019**, *127*, 282–294.
- (16) Nian, R.; Liu, J.; Huang, B. A review on reinforcement learning: Introduction and applications in industrial process control. *Comput. Chem. Eng.* **2020**, *139*, 106886.
- (17) Devarakonda, V. S.; Sun, W.; Tang, X.; Tian, Y. Recent Advances in Reinforcement Learning for Chemical Process Control. *Processes* **2025**, *13*, 1791.
- (18) Bloor, M.; Ahmed, A.; Kotecha, N.; Mercangoz, M.; Tsay, C.; del Rio-Chanona, E. A. Control-informed reinforcement learning for chemical processes. *Ind. Eng. Chem. Res.* **2025**, *64*, 4966–4978.
- (19) Bangi, M. S. F.; Kwon, J. S.-I. Deep reinforcement learning control of hydraulic fracturing. *Comput. Chem. Eng.* **2021**, *154*, 107489.
- (20) Khodaverdian, A.; Cui, X.; Christofides, P. D. Utilizing reinforcement learning in feedback control of nonlinear processes with stability guarantees. *Digital Chem. Eng.* **2025**, *17*, 100277.
- (21) Gerold, H.; Lucia, S. Safe reinforcement learning via adaptive robust model predictive shielding. *Comput. Chem. Eng.* **2026**, *206*, 109521.
- (22) Brandi, S.; Piscitelli, M. S.; Martellacci, M.; Capozzoli, A. Deep reinforcement learning to optimize indoor temperature control and heating energy consumption in buildings. *Energy and Buildings* **2020**, *224*, 110225.
- (23) Zou, Z.; Yu, X.; Ergun, S. Towards optimal control of air handling units using deep reinforcement learning and recurrent neural network. *Build. Environ.* **2020**, *168*, 106535.
- (24) Yerimah, L. E.; Jorgensen, C.; Bequette, B. W. Model-based policy optimization algorithms for feedback control of complex dynamic systems. *Comput. Chem. Eng.* **2025**, *195*, 109032.
- (25) Hassanpour, H.; Wang, X.; Corbett, B.; Mhaskar, P. A practically implementable reinforcement learning-based process controller design. *AIChE J.* **2024**, *70*, No. e18245.
- (26) Lewis, F. L.; Vrabie, D.; Syrmos, V. L. *Optimal Control*; John Wiley & Sons, 2012.
- (27) Lawrence, N. P.; Mesbah, A. Why Goal-Conditioned Reinforcement Learning Works: Relation to Dual Control. 2025; *arXiv*; arXiv:2512.06471.
- (28) Engel, J.-M.; Babuška, R. On-line reinforcement learning for nonlinear motion control: Quadratic and non-quadratic reward functions. *IFAC Proceedings Volumes* **2014**, *47*, 7043–7048.
- (29) Weber, D.; Schenke, M.; Wallscheid, O. Steady-state error compensation for reinforcement learning-based control of power electronic systems. *IEEE Access* **2023**, *11*, 76524–76536.
- (30) Tracey, B. D.; Michi, A.; Chervonyi, Y.; Davies, I.; Paduraru, C.; Lazic, N.; Felici, F.; Ewalds, T.; Donner, C.; Galperti, C.; et al. others Towards practical reinforcement learning for tokamak magnetic control. *Fusion Eng. Des.* **2024**, *200*, 114161.
- (31) Chen, D.; Peng, Y.; Zheng, T.; Wang, H.; Qu, C.; Cheng, L. Adviser-Actor-Critic: Eliminating Steady-State Error in Reinforcement Learning Control. 2025. *arXiv*, arXiv:2502.02265..
- (32) Burda, Y.; Edwards, H.; Storkey, A.; Klimov, O. Exploration by Random Network Distillation. In *International Conference on Learning Representations*, 2019. <https://openreview.net/forum?id=H1JJnRSYm>.
- (33) Hu, Y.; Wang, W.; Jia, H.; Wang, Y.; Chen, Y.; Hao, J.; Wu, F.; Fan, C. Learning to utilize shaping rewards: A new approach of reward shaping. In *Advances in Neural Information Processing Systems*, 2020; Vol. 33, pp 15931–15941.
- (34) Salerno, D.; Arellano-Garcia, H.; Wozny, G. Ethylene separation by feed-splitting from light gases. *Energy* **2011**, *36*, 4518–4523.
- (35) Jalanko, M.; Sanchez, Y.; Mahalec, V.; Mhaskar, P. Adaptive system identification of industrial ethylene splitter: A comparison of subspace identification and artificial neural networks. *Comput. Chem. Eng.* **2021**, *147*, 107240.
- (36) Leegwater, H. *Practical Distillation Control*; Springer, 1992; pp 331–350.
- (37) Muske, K. R.; Badgwell, T. A. Disturbance modeling for offset-free linear model predictive control. *J. Process Control* **2002**, *12*, 617–632.
- (38) Pannocchia, G.; Rawlings, J. B. Disturbance models for offset-free model-predictive control. *AIChE J.* **2003**, *49*, 426–437.
- (39) Barto, A. G. Reinforcement learning: An introduction. by richard's sutton. *SIAM Rev.* **2021**, *6*, 423.
- (40) Joshi, T.; Makker, S.; Kodamana, H.; Kandath, H. Twin actor twin delayed deep deterministic policy gradient (TATD3) learning for batch process control. *Comput. Chem. Eng.* **2021**, *155*, 107527.
- (41) Fang, J.; Zheng, Q.; Cai, C.; Chen, H.; Zhang, H. Deep reinforcement learning method for turbofan engine acceleration optimization problem within full flight envelope. *Aerosp. Sci. Technol.* **2023**, *136*, 108228.
- (42) Silver, D.; Lever, G.; Heess, N.; Degris, T.; Wierstra, D.; Riedmiller, M. Deterministic policy gradient algorithms. In *International conference on machine learning*, 2014; pp 387–395.
- (43) Lillicrap, T. P.; Hunt, J. J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; Wierstra, D. Continuous Control with Deep Reinforcement Learning. In *International Conference on Learning Representations*, 2016, ICLR poster record listed by DBLP. <https://dblp.org/rec/journals/corr/LillicrapHPHETS15>.
- (44) Fujimoto, S.; Hoof, H.; Meger, D. Addressing function approximation error in actor-critic methods. In *International conference on machine learning*, 2018; pp 1587–1596.
- (45) Wang, L.; Zheng, Z.; Lin, Y. Steady-State Error Compensation for Reinforcement Learning with Quadratic Rewards. In *14th Asian Control Conference (ASCC)*, 2024, pp 1608–1613.
- (46) Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. Prioritized Experience Replay. *arXiv* **2016**, arXiv:1511.05952.
- (47) Li, D.; Jacobsen, A.; White, A. Revisiting experience replay in non-stationary environments. In *International conference on autonomous agents and multiagent systems*, 2021.
- (48) Fedus, W.; Ramachandran, P.; Agarwal, R.; Bengio, Y.; Larochelle, H.; Rowland, M.; Dabney, W. Revisiting fundamentals of experience replay. In *International conference on machine learning*, 2020; pp 3061–3071.
- (49) Zhang, S.; Sutton, R. S. A Deeper Look at Experience Replay. *arXiv* **2018**, arXiv:1712.01275.
- (50) Bustos, G. A.; Ferramosca, A.; Godoy, J. L.; González, A. H. Application of model predictive control suitable for closed-loop re-identification to a polymerization reactor. *J. Process Control* **2016**, *44*, 1–13.